

层次结构存储系统

计算机采用“存储程序”工作方式，意味着在程序执行时所有指令和数据都是从存储器中取出来执行的。存储器是计算机系统中的重要组成部分，相当于计算机的“仓库”，用来存放各类程序及其处理的数据。计算机中所用的存储元件有多种类型，如触发器构成的寄存器、半导体静态 RAM 和动态 RAM、Flash 存储器、磁盘、磁带和光盘等，它们各自有不同的速度、容量和价格，各类存储器按照层次化方式构成计算机的存储器整体结构。

本章主要介绍构成层次化存储结构的几类存储器的工作原理和组织形式。主要包括：半导体随机存取存储器、只读存储器、Flash 存储器、磁盘存储器等不同类型存储器的特点，存储器芯片和 CPU 的连接，高速缓存的基本原理以及虚拟存储器系统的实现技术等。

6.1 存储器概述

6.1.1 存储器的分类

根据存储器的特点和使用方法的不同，可以有以下几种分类方法。

1. 按存储元件分类

存储元件必须具有截然不同且相对稳定的两个物理状态，才能被用来表示二进制代码 0 和 1。目前使用的存储元件主要有半导体器件、磁性材料和光介质。用半导体器件构成的存储器称为半导体存储器；磁性材料存储器主要是磁表面存储器，如磁盘存储器和磁带存储器；光介质存储器称为光盘存储器。

2. 按存取方式分类

(1) 随机存取存储器

随机存取存储器（Random Access Memory，简称 RAM）的特点是按地址访问存储单元，因为每个地址译码时间相同，所以，在不考虑芯片内部缓冲的前提下，每个单元的访问时间是一个常数，与地址无关。不过，现在的 DRAM 芯片内都具有行缓冲，因而有些数据可能因为已在行缓冲中而缩短了访问时间。随机存取存储器的存储介质是半导体存储器件。

(2) 顺序存取存储器

顺序存取存储器（Sequential Access Memory，简称 SAM）的特点是信息按顺序存放和读出，其存取时间取决于信息存放位置，以记录块为单位编址。磁带存储器就是一种顺序存取存

储器，其存储容量大，但存取速度慢。

(3) 直接存取存储器

直接存取存储器 (Direct Access Memory, 简称 DAM) 的存取方式兼有随机访问和顺序访问的特点。首先直接定位到需读写信息所在区域的开始处，然后按顺序方式存取，磁盘存储器就是如此。

(4) 相联存储器

上述三类存储器都是按所需信息的地址来访问，但有些情况下可能不知道所访问信息的地址，只知道要访问信息的内容特征，此时，只能按内容检索到存储位置进行读写。这种存储器称为按内容访问存储器 (Content Addressed Memory, 简称 CAM) 或相联存储器 (Associative Memory, 简称 AM)。如快表就是一种相联存储器。

3. 按信息的可更改性分类

按信息可更改性分读写存储器 (Read/Write Memory) 和只读存储器 (Read Only Memory)。读写存储器中的信息可以读出和写入，RAM 芯片是一种读写存储器；只读存储器用 ROM 表示，ROM 存储器芯片中的信息一旦确定，通常在联机情况下只能读不能写，但在某些情况下也可重新写入。

RAM 芯片和 ROM 芯片都采用随机存取方式进行信息的访问。

4. 按断电后信息的可保存性分类

按断电后信息的可保存性分成非易失性存储器 (Nonvolatile Memory) 和易失性存储器 (Volatile Memory)。非易失性存储器也称不挥发性存储器，其信息可一直保留，不需电源维持，例如，ROM、磁表面存储器、光盘存储器等都是非易失性存储器；易失性存储器也称挥发性存储器，在电源关闭时信息自动丢失，例如，RAM、cache 等都是易失性存储器。

5. 按功能分类

(1) 高速缓冲存储器

高速缓冲存储器 (cache) 简称高速缓存，位于主存和 CPU 之间，目前主要由静态 RAM 芯片组成，其存取速度接近 CPU 的工作速度，用来存放当前 CPU 经常使用到的指令和数据。

(2) 主存储器

指令直接面向的存储器是主存储器，简称主存。CPU 执行指令时给出的存储地址最终必须转换为主存地址，若不采用虚拟存储管理，则 CPU 直接给出主存地址。主存是存储器分层结构中的核心存储器，用来存放系统中启动运行的程序及其数据，主存目前一般用 MOS 管半导体存储器构成。

(3) 辅助存储器

把系统运行时直接和主存交换信息的存储器称为辅助存储器，简称辅存。磁盘存储器比磁带和光盘存储器速度快，因此，目前大多用磁盘存储器作为辅存，辅存的内容需要调入主存后才能被 CPU 访问。

(4) 海量后备存储器

磁带存储器和光盘存储器的容量大、速度慢，主要用于信息的备份和脱机存档，因此被用作海量后备存储器。

辅存和海量后备存储器统称为外部存储器，简称外存。

6.1.2 主存储器的组成和基本操作

如图 6.1 所示是主存储器（Main Memory，简称 MM）的基本框图。其中由一个个存储 0 或 1 的记忆单元（cell）构成的存储阵列是存储器的核心部分。这种记忆单元也称为存储元、位元，它是具有两种稳态的能表示二进制 0 和 1 的物理器件。存储阵列（bank）也称为存储体、存储矩阵。为了存取存储体中的信息，必须对存储单元编号，所编号码就是地址。编址单位（addressing unit）是指具有相同地址的那些位元构成的一个单位，可以是一个字节或一个字。对各存储单元进行编号的方式称为编址方式（addressing mode），可以按字节编址，也可以按字编址。现在大多数通用计算机都采用字节编址方式，此时，存储体内一个地址中有一个字节。也有许多专用于科学计算的大型计算机采用 64 位编址，这是因为科学计算中数据大多是 64 位浮点数。

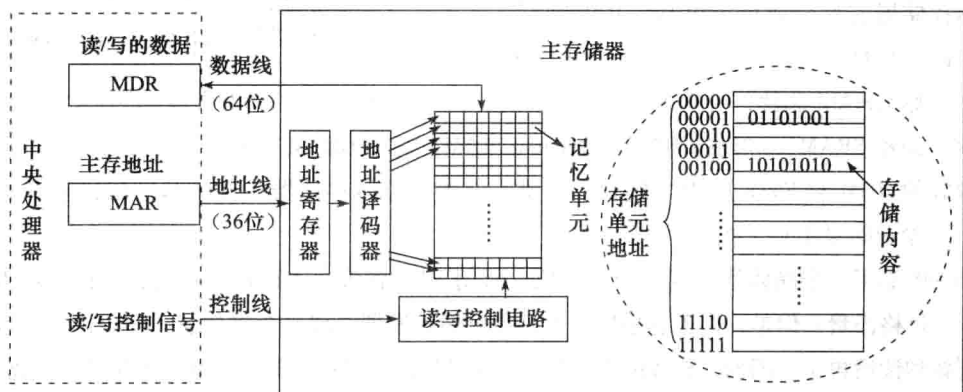


图 6.1 主存储器基本框图

如图 6.1 所示，指令执行过程中需要访问主存时，CPU 首先把需访问单元的地址送到主存地址寄存器（Memory Address Register，简称 MAR）中，然后通过地址线将主存地址送到主存中的地址寄存器，以便地址译码器进行译码后选中相应单元，同时，CPU 将读/写控制信号通过控制线送到主存的读写控制电路。如果是写操作，CPU 同时将要写的信息送主存数据寄存器（Memory Data Register，简称 MDR）中，在读写控制电路的控制下，经数据线将信息写入选中的单元；如果是读操作，则主存读出选中单元的内容送数据线，然后送到 MDR 中。数据线的宽度与 MDR 的宽度相同，地址线的宽度与 MAR 的宽度相同。图中采用 64 位数据线，因此，在字节编址方式下，每次最多可以存取 8 个字节的内容。地址线的位数决定了主存地址空间的最大可寻址范围，例如，36 位地址的最大可寻址范围为 $0 \sim 2^{36} - 1$ 。注意：在计算机中所有地址的编号总是从 0 开始。

6.1.3 存储器的主要性能指标

虽然在计算机出现至今的几十年内，存储器介质和特性有了很大变化，但评价其性能的主要指标仍然是容量、速度和价格。

存储器速度可用访问时间、存储周期或存储器带宽来表示。**访问时间**一般用读出时间 T_A 及写入时间 T_W 来描述。**读出时间** T_A 是指从存储器接到读命令开始至信息被送到数据线上所需的时间；**写入时间** T_W 是指存储器接到写命令开始至信息被写入存储器所需的时间。**存储周期**是指存储器进行一次读写操作所需要的全部时间，也就是存储器进行连续读写操作所允许的最短间隔时间，它应等于访问时间加上下一次存取开始前所要求的附加时间，一般用 T_M 表示。存储器中由于读出放大器、驱动电路等都有一段稳定恢复时间，读出后不能立即进行下一次访问，所以，一般 T_M 、 T_A 和 T_W 存在以下关系： $T_M > T_A$ 、 $T_M > T_W$ 。**存储器带宽** B 表示存储器被连续访问时可以提供的数据传送速率，通常用每秒钟传送信息的位数（或字节数）来衡量。

6.1.4 各类存储元件的特点

目前使用的存储器基本元件主要有半导体器件、磁性材料和光介质。如图 6.2 所示，半导体存储器芯片分成可读可写（RWM）芯片和只读（ROM）芯片两大类。可读可写芯片习惯上称为 **RAM 芯片**，分 **静态 RAM**（Static RAM，简称 SRAM）和 **动态 RAM**（Dynamic RAM，简称 DRAM）两种。只读芯片有不可在线改写内容的 ROM 和 Flash ROM（闪存）两类。有关半导体存储器元件的基本结构和基本读写原理，请参看附录中相关内容。

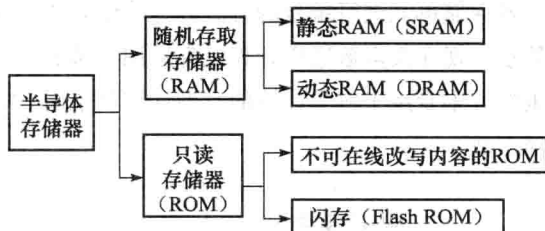


图 6.2 半导体存储器芯片类型

SRAM 要用 6 个晶体管实现一个二进位，所用 MOS 管多，占硅片面积大，因而功耗大，集成度低，价格昂贵；但是，由于它采用一个正负反馈触发器电路来存储信息，所以读写速度快且无需刷新和读后再生。因而，SRAM 适合做高速小容量的半导体存储器，如高速缓冲存储器。

DRAM 只要一个晶体管就能实现一个二进位，所用 MOS 管少，占硅片面积小，因而功耗小，集成度高，价格相对便宜；但是，由于它采用电容储存电荷来存储信息，所以速度较慢，且必须定时刷新和读后再生。因而 DRAM 适合做慢速大容量的半导体存储器，如主存储器。

不可在线改写内容的 ROM 有多种类型，可分为 MROM、PROM、EPROM 和 EEPROM（E²PROM）等。这种芯片与 RAM 芯片一样，也以随机存取方式工作；信息用特殊方式写入，一经写入，就可长久保存，不受断电影响，故这种芯片做成的存储器是非易失性存储器。这类存储器用来存放一些固定程序，如监控程序、启动程序等；也可作为控制存储器，存放微程序；还可作为函数发生器和代码转换器；在输入/输出设备中，被用作字符发生器，汉字库等；在嵌入式设备中用来存放固化的程序。

Flash 存储器也称为**闪存**，是高密度非易失性读写存储器，它兼有 RAM 和 ROM 的优点，而且功耗低、集成度高，不需后备电源。这种器件可在计算机内进行擦除和编程写入，因此又称为**快擦型电可擦除重编程 ROM**。目前被广泛使用的 U 盘和存储卡等都属于 Flash 存储器，也用于存放 BIOS。

磁表面存储器中信息的存取主要由磁层和磁头来完成。磁层是存放信息的介质，磁头是实现“磁-电”和“电-磁”转换的元件。磁表面存储器读写时，一般使磁头固定，而磁层（载磁体）做高速回转或匀速直线运动。在这种相对运动中，通过磁头进行信息存取。因此，

其信息存取过程属于机械运动，速度很慢。目前，DRAM 的读写速度比磁盘快 10 ~ 100 万倍，SRAM 比磁盘快 100 万倍以上；但是一个磁盘的容量则是一个 DRAM 芯片的几百到几千倍。

6.1.5 存储器的层次结构

存储器容量和性能应随着处理器速度和性能的提高而同步提高，以保持系统性能的平衡。然而，在过去 20 多年中，随着时间的推移，处理器和存储器在性能发展上的差异越来越大，存储器在容量尤其是访问延时方面的性能增长越来越跟不上处理器性能发展的需要。为了缩小存储器和处理器两者之间在性能方面的差距，通常在计算机内部采用层次化的存储器体系结构。

某一种元件制造的存储器很难同时满足大容量、高速度和低成本的要求。比如 SRAM 的存取速度快，但是难以构成大容量存储器；而大容量、低成本的磁表面存储器的存取速度又远远低于半导体存储器，并且难以实现随机存取。因此，在计算机中有必要把各种不同容量和不同存取速度的存储器按一定的结构有机地组织在一起，形成层次化的存储器结构，把程序和数据按不同的层次存放在各级存储器中，使得整个存储系统在速度、容量和价格等方面具有较好的综合性能指标。图 6.3 是存储系统层次结构示意图，其中列出的典型存取时间和存储容量是过去某个时间点的情况，它们会随着时间而变化，不过，这些数据反映出来的速度和容量之间的关系通常不会随着时间而发生大的变化。

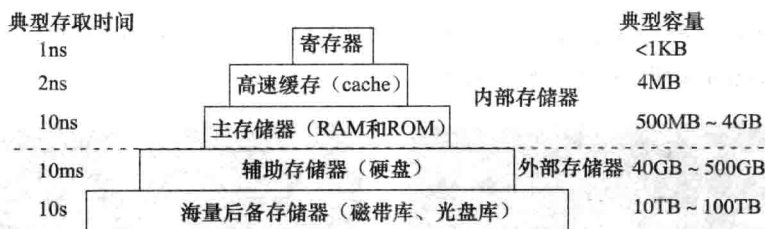


图 6.3 存储器层次化体系结构示意图

从图 6.3 中可以看出，速度越快则容量越小、越靠近 CPU。CPU 可以直接访问内部存储器，而外部存储器的信息则要先取到主存，然后才能被 CPU 访问。CPU 执行指令时，需要的操作数大部分都来自寄存器；当需要从（向）存储器中取（存）数据时，先访问 cache，如果不在 cache 中，则访问主存，如果不在主存中，则访问硬盘，此时，操作数从硬盘中读出送到主存，然后从主存送到 cache。

数据使用时一般只在相邻两层之间复制传送，而且总是从慢速存储器复制到快速存储器。传送的单位是一个定长块，因此需要确定定长块的大小，并在相邻两层间建立块之间的映射关系。

6.2 主存与 CPU 的连接及其读写操作

6.2.1 主存模块的连接和读写操作

主存与 CPU 之间的连接如图 6.4 所示。CPU 通过其芯片内的总线接口部件（即总线控制

逻辑)与系统总线^②相连,然后再通过总线之间的 I/O 桥接器、存储器总线连接到主存。

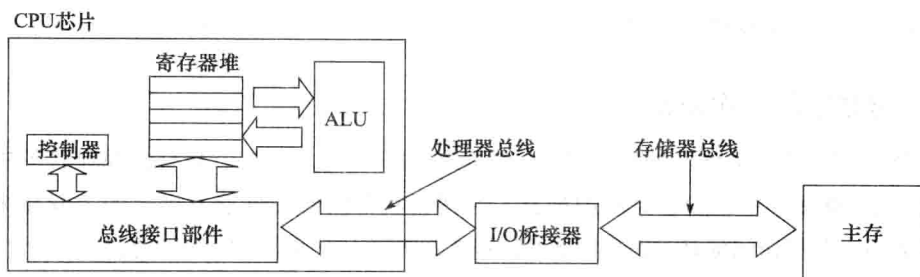
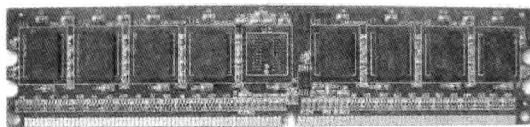


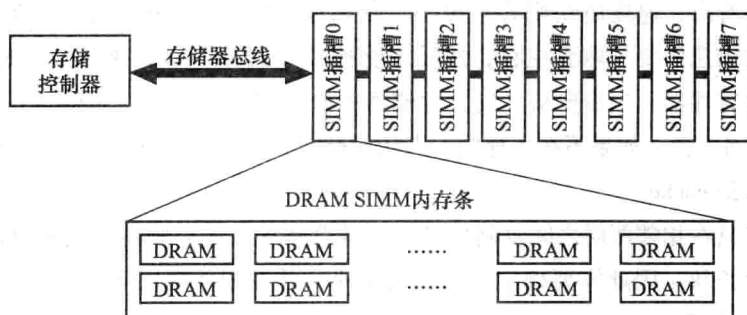
图 6.4 主存与 CPU 的连接

总线是连接其上的各部件共享的传输介质,通常由控制线、数据线和地址线构成。如图 6.4 所示,计算机中各部件之间通过总线相连,例如,CPU 通过系统总线和存储器总线与主存相连。在 CPU 和主存之间交换信息时,CPU 通过总线接口部件把地址信息和总线控制信息分别送到地址线和控制线,CPU 和主存之间交换的数据则通过数据线传输。

受集成度和功耗等因素的限制,单个芯片的容量不可能很大,所以往往通过存储器芯片扩展技术,将多个芯片做在一个内存模块(即内存条)上,然后由多个内存模块以及主板或扩充板上的 RAM 芯片和 ROM 芯片组成一台计算机所需的主存空间,再通过总线、桥接器等和 CPU 相连,如图 6.5 所示。图 6.5a 是内存条和内存条插槽(slot)示意图,图 6.5b 是存储控制器(memory controller)、存储器总线、内存条和 DRAM 芯片之间的连接关系示意图。存储控制器可以包含在图 6.4 所示的 I/O 桥接器中。



a) 内存条和内存条插槽



b) 存储控制器、存储器总线、内存条和 DRAM 芯片之间的连接

图 6.5 DRAM 芯片在系统中的位置及其连接关系

② 国内教材中系统总线通常指连接 CPU、存储器和各种 I/O 模块等主要部件的总线统称,而 Intel 公司推出的芯片组中,对系统总线赋予了特定的含义,特指 CPU 连接到北桥芯片的总线,也称为处理器总线或前端总线(Front Side Bus,简称 FSB)。

由若干个存储器芯片构成一个存储器时,需要在字方向和位方向上进行扩展。位扩展指用若干片位数较少的存储器芯片构成给定字长的存储器。例如,用8片 $4\text{K} \times 1$ 位的芯片构成 $4\text{K} \times 8$ 位的存储器,需在位方向上扩展8倍,而字方向上无需扩展。字扩展是容量的扩充,位数不变。例如,用 $16\text{K} \times 8$ 位的存储芯片在字方向上扩展4倍,构成一个 $64\text{K} \times 8$ 位的存储器。当芯片在容量和位数上都不满足存储器要求的情况下,需要对字和位同时扩展。例如,用 $16\text{K} \times 4$ 位的存储器芯片在字方向上扩展4倍、位方向上扩展2倍,可构成一个 $64\text{K} \times 8$ 位的存储器。

如图6.6所示,给出了用8个 $16\text{M} \times 8$ 位的DRAM芯片扩展构成一个128MB内存条的示意图。每片DRAM芯片中有一个 $4096 \times 4096 \times 8$ 位的存储阵列,所以,行地址和列地址各12位($2^{12} = 4096$),有8个位平面。

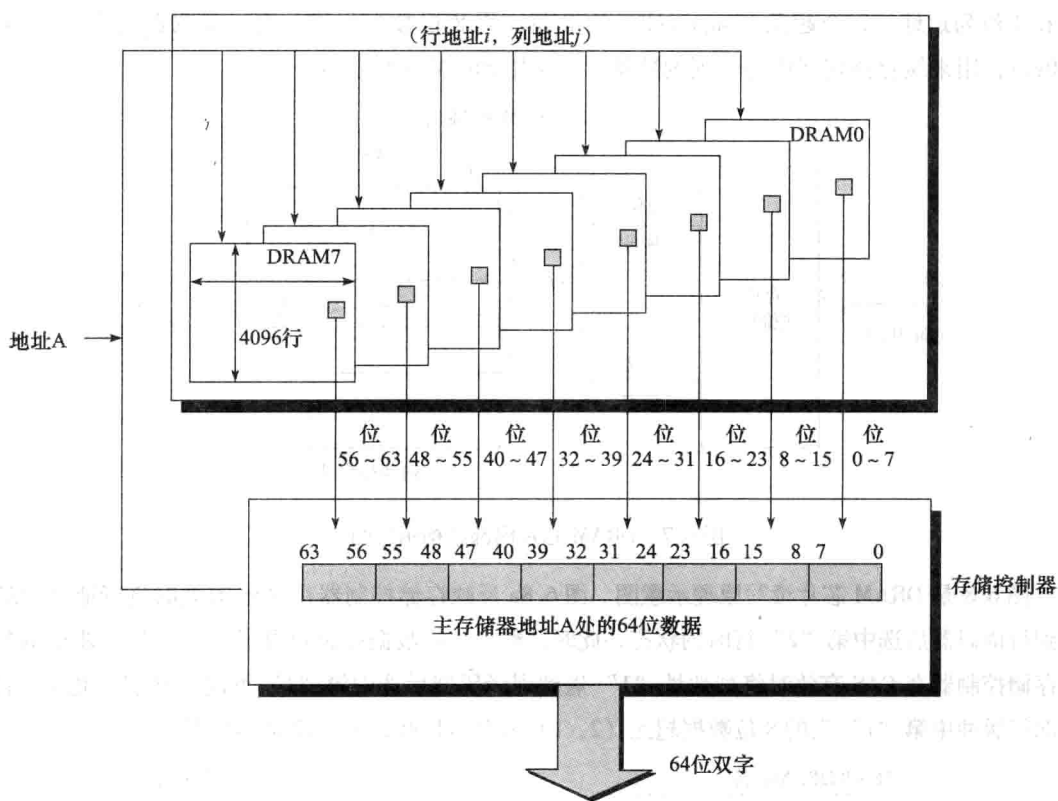


图6.6 DRAM 芯片的扩展

内存条通过存储器总线连接到存储控制器,CPU通过存储控制器对内存条中的DRAM芯片进行读写,CPU要读写的存储单元地址通过总线被送到存储控制器,然后由存储控制器将存储单元地址转换为DRAM芯片的行地址 i 和列地址 j ,分别在行地址选通信号(RAS)和列地址选通信号(CAS)的控制下通过DRAM芯片的地址引脚分时送到DRAM芯片内部的行地址译码器和列地址译码器,以选择行、列地址交叉点(i, j)的8位数据同时进行读写,8个芯片可组合成需要的传输宽度(如64位),再通过存储器总线进行传输。

在图6.6所示的存储器结构中,第0~7单元可以同时读写,第8~15单元可以同时读写,以此类推。若访问的数据在第6、7、8、9这4个存储单元中,则需要访问两次存储器。了解

存储器结构可以更好地理解第3章提到的数据对齐问题。

若一个 $2^n \times b$ 位 DRAM 芯片的存储阵列是 r 行 $\times$ c 列, 则该芯片容量为 $2^n \times b$ 位且 $2^n = r \times c$, 芯片内的地址位数为 n , 其中行地址位数为 $\log_2 r$, 列地址位数为 $\log_2 c$, n 位地址中高位部分为行地址, 低位部分为列地址, 为提高 DRAM 芯片的性价比, 通常设置的 r 和 c 满足 $r \leq c$ 且 $|r - c|$ 最小。例如, 对于 $8K \times 8$ 位 DRAM 芯片, 其存储阵列设置为 2^6 行 $\times$ 2^7 列, 因此行地址和列地址的位数分别为 6 位和 7 位, 13 位芯片内地址 $A_{12}A_{11} \cdots A_1A_0$ 中, 行地址为 $A_{12}A_{11} \cdots A_7$, 列地址为 $A_6 \cdots A_1A_0$ 。

图 6.7 是 DRAM 芯片内部结构示意图。图中芯片容量为 16×8 位, 存储阵列为 4 行 $\times$ 4 列, 地址引脚采用复用方式, 因而仅需 2 根地址引脚, 在 RAS 和 CAS 的控制下分时传送 2 位行地址和 2 位列地址。每个超元 (supercell) 有 8 位, 需 8 根数据引脚, 有一个内部行缓冲 (row buffer), 用来缓存指定行中每一列的数据, 通常用 SRAM 元件实现。

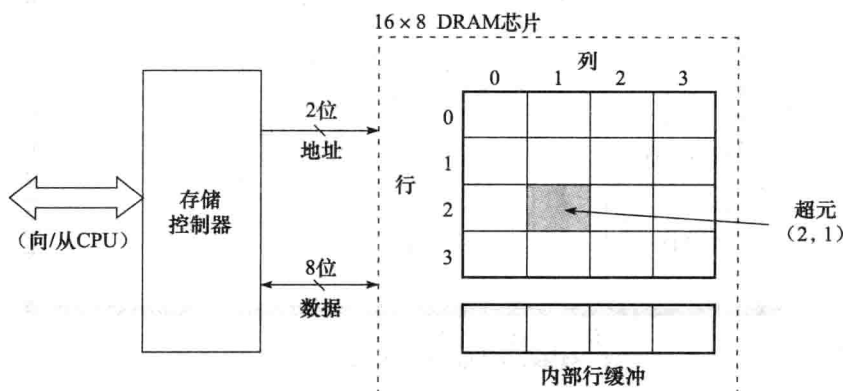


图 6.7 DRAM 芯片内部结构示意图

图 6.8 是 DRAM 芯片读写原理示意图。图 6.8a 反映存储控制器在 RAS 有效时将行地址“2”送到行译码器后选中第“2”行时的状态, 此时, 整个一行数据被送到内部行缓冲中。图 6.8b 反映存储控制器在 CAS 有效时将列地址“1”送到列译码器后选中第“1”列时的状态, 此时, 将内部行缓冲中第“1”列的 8 位数据超元 (2, 1) 读到数据线上, 并继续向 CPU 传送。

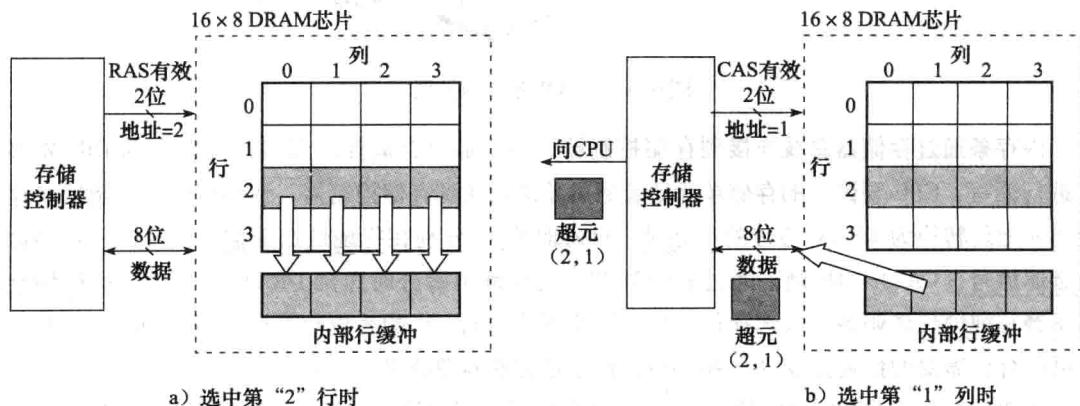


图 6.8 DRAM 芯片读写原理示意图

6.2.2 “装入”指令和“存储”指令操作过程

访存指令主要有两类：装入（load）指令用于将存储单元内容装入 CPU 的寄存器中，如 IA-32 中的“`movl 8(%ebp), %eax`”指令等；存储（store）指令用于将 CPU 寄存器内容存储到存储单元中，如 IA-32 中的“`movl %eax, 8(%ebp)`”指令等。

假定装入指令“`movl 8(%ebp), %eax`”中存储操作数“`8(%ebp)`”对应的主存地址为 A，则取数过程如图 6.9 所示。

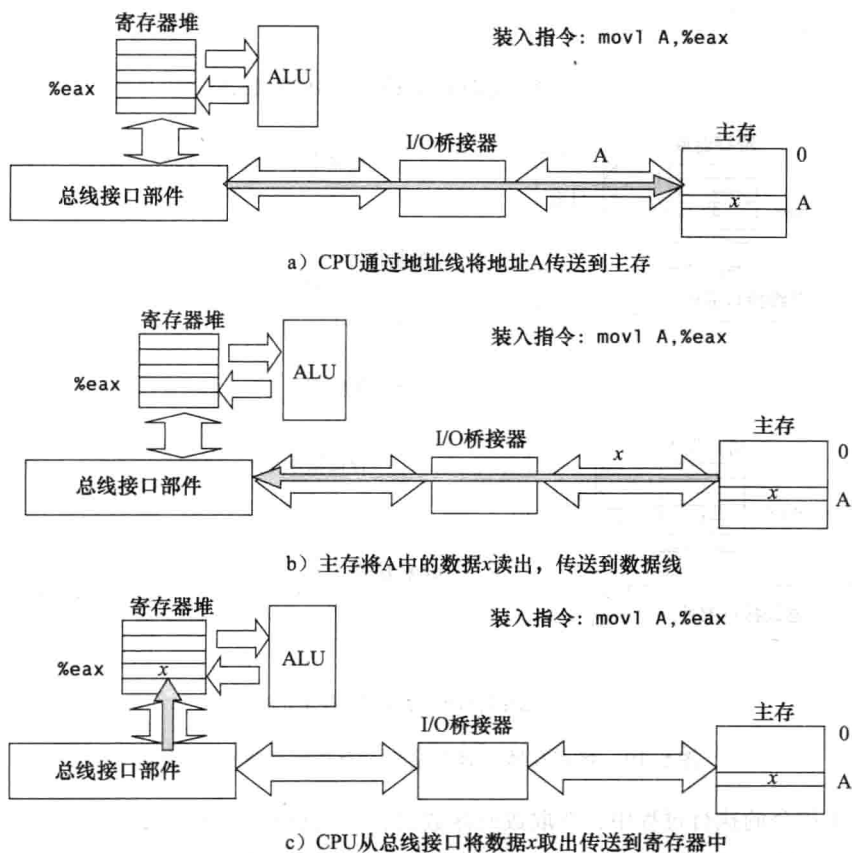


图 6.9 从主存单元取数到寄存器的操作过程

图 6.9a 表示取数操作第一步，CPU 将主存地址 A 通过总线接口送到地址线，然后由存储控制器将地址 A 分解成行、列地址按分时方式送 DRAM 芯片；图 6.9b 表示取数操作第二步，主存将地址 A 中的数据 `x` 通过数据线送到总线接口部件中；图 6.9c 表示取数操作第三步，CPU 从总线接口部件中取出 `x` 存放到寄存器 EAX 中。实际上，上述过程的第一步同时还会把“存储器读”控制命令通过控制线送到主存，这在图 6.9a 中没有表示出来。

假定存储指令“`movl %eax, 8(%ebp)`”中主存操作数“`8(%ebp)`”的主存地址为 A，则存数操作过程如图 6.10 所示。

图 6.10a 表示存数操作第一步，其过程与图 6.9a 相同；图 6.10b 表示存数操作第二步，CPU 将寄存器 EAX 中的数据 `y` 通过总线接口部件送到数据线；图 6.10c 表示存数操作第三步，

主存将数据线上的 y 存到主存单元 A 中。实际上,上述过程的第一步同时还会把“存储器写”控制命令通过控制线送到主存,这在图 6.10a 中没有表示出来,而且,第二步将数据 y 送数据线也可以和第一步同时进行。

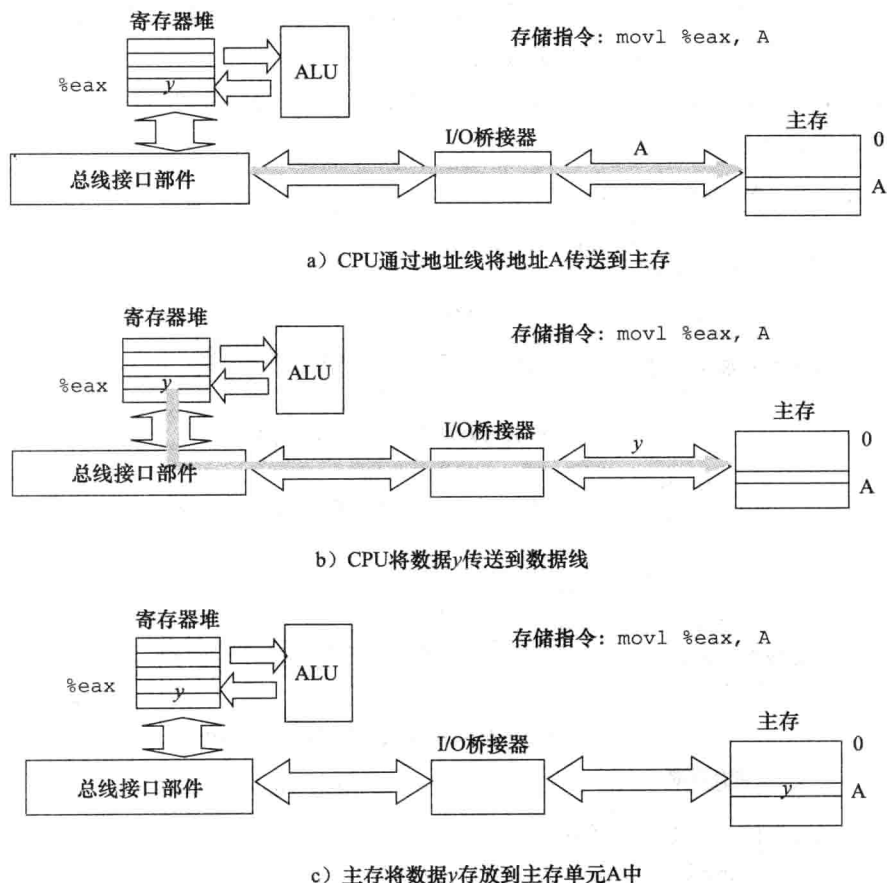


图 6.10 将寄存器内容存储到主存单元的操作过程

上述两条指令的执行过程中,在取数或存数前,需要根据“ $8(\%ebp)$ ”计算主存地址 A ,其计算操作在 CPU 中完成,它涉及 IA-32 中的分段和分页存储管理机制,具体过程将在 6.6 节中说明。

6.3 磁盘存储器

6.3.1 磁盘存储器的结构

磁盘存储器主要由磁记录介质、磁盘驱动器、磁盘控制器三大部分组成。磁盘控制器 (disk controller) 包括控制逻辑、时序电路、“并→串”转换和“串→并”转换电路。磁盘驱动器包括读写电路、读/写转换开关、读/写磁头与磁头定位伺服系统。图 6.11 是磁盘驱动器的物理组成示意图。

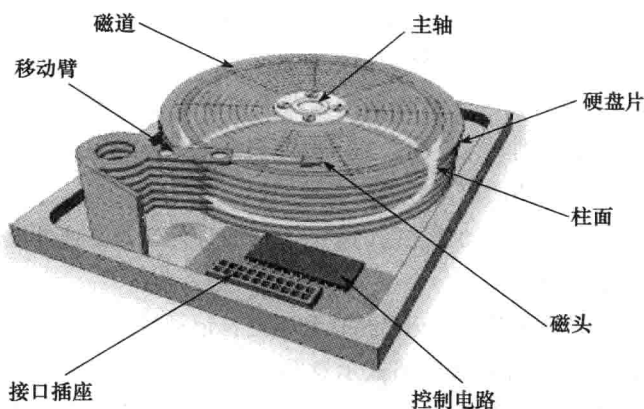


图 6.11 磁盘驱动器的物理组成

如图 6.11 所示，磁盘驱动器主要由多张硬盘片、主轴、主轴电机、移动臂、磁头和控制电路等部分组成，通过接口与磁盘控制器连接，每个盘片的两个面上各有一个磁头，因此，磁头号就是盘面号。磁头和盘片相对运动形成的圆构成一个磁道（track），磁头位于不同的半径上，则得到不同的磁道。多个盘片上相同磁道形成一个柱面（cylinder），所以，磁道号就是柱面号。信息存储在盘面的磁道上，而每个磁道被分成若干扇区（sector），以扇区为单位进行磁盘读写。在读写磁盘时，总是写完一个柱面上所有的磁道后，再移到下一个柱面。磁道从外向里编址，最外面的为磁道 0。

图 6.12 所示是磁盘驱动器的内部逻辑。

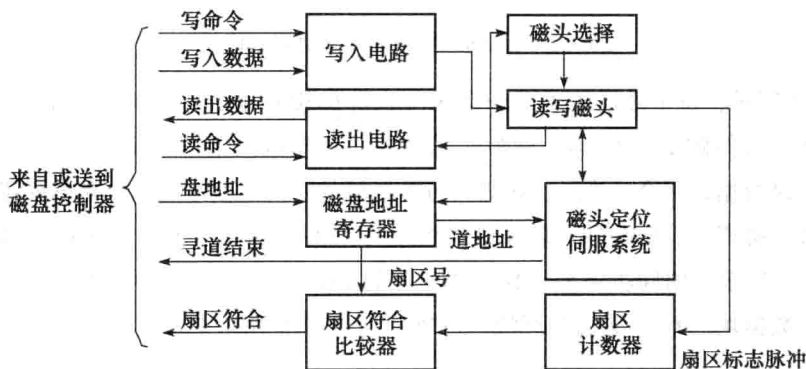


图 6.12 磁盘驱动器的内部逻辑结构

磁盘读写是指根据主机访问控制字中的盘地址（磁头号、柱面号、扇区号）读写目标磁道中的指定扇区。因此，其操作可归纳为寻道、旋转等待和读写三个步骤。如图 6.12 所示的操作过程如下。

① 寻道操作：磁盘控制器把盘地址送到磁盘驱动器的磁盘地址寄存器后，便产生寻道命令，启动磁头定位伺服系统，根据磁头号和柱面号，选择指定的磁头移动到指定的柱面。此操作完成后，发出寻道结束信号给磁盘控制器，并转入旋转等待操作。

② 旋转等待操作：盘片旋转时，首先将扇区计数器清零，以后每来一个扇区标志脉冲，

扇区计数器加1, 把计数内容与磁盘地址寄存器中的扇区地址进行比较, 如果一致, 则输出扇区符合信号, 说明要读写的信息已经转到磁头下方。

③ **读写操作**: 扇区符合信号送给磁盘控制器后, 磁盘控制器的读写控制电路开始动作。如果是写操作, 就将数据送到写入电路, 写入电路根据记录方式生成相应的写电流脉冲; 如果是读操作, 则由读出放大电路读出内容送磁盘控制器。

磁盘控制器是主机与磁盘驱动器之间的接口。磁盘存储器是高速外设, 所以磁盘控制器和主机之间采用成批数据交换方式。

数据在磁盘上的记录格式分定长记录格式和不定长记录格式两种。目前大多采用定长记录格式。图 6.13 是温切斯特磁盘的磁道格式示意图, 它采用定长记录格式。最早的硬盘由 IBM 公司开发, 称为温切斯特盘 (Winchester 是一个地名), 简称温盘, 它是几乎所有现代硬盘产品的原型。

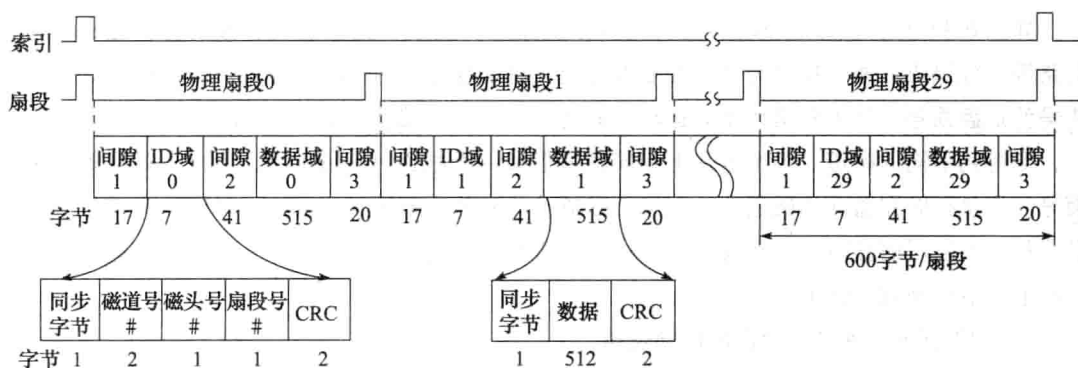


图 6.13 温切斯特磁盘的磁道记录格式

如图 6.13 所示, 每个磁道由若干个扇区 (也称扇段) 组成, 每个扇区记录一个数据块, 每个扇区由头空 (间隙1)、ID 域、间隙 (间隙2)、数据域和尾空 (间隙3) 组成。头空占 17 个字节, 不记录数据, 用全 1 表示, 磁盘转过该区域的时间是留给磁盘控制器作准备用的; ID 域由同步字节、磁道号、磁头号、扇段号和相应的 CRC 码组成, 同步字节标志 ID 域的开始; 数据域占 515 个字节, 由同步字节、数据和相应的 CRC 码组成, 其中真正的数据区占 512 字节; 尾空是在数据块的 CRC 码后的区域, 占 20 个字节, 也用全 1 表示。

6.3.2 磁盘存储器的性能指标

(1) 记录密度

记录密度可用道密度和位密度来表示。在沿磁道分布方向上, 单位长度内的磁道数目叫道密度。在沿磁道方向上, 单位长度内存放的二进制信息数目叫位密度。如图 6.14 所示是磁盘盘面上的道密度和位密度示意图。左边采用的是低密度存储方式, 所有磁道上的扇区数相同, 所以每个磁道上的位数相同, 因而内道上的位密度比外道位密度高; 右边采用的是高密度存储方式, 每个磁道上的位密度相同, 所以外道上的扇区数比内道上扇区数多, 因而整个磁盘的容量比低密度盘高得多。

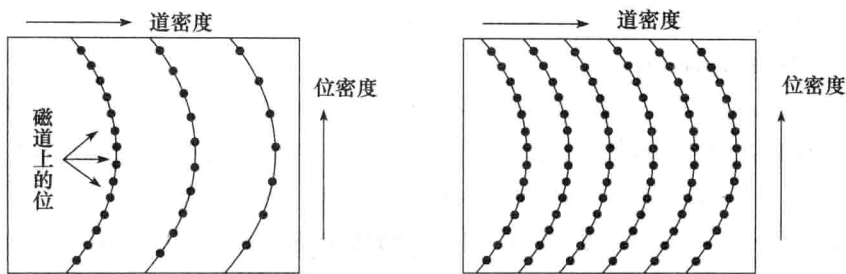


图 6.14 磁盘盘面上的记录密度示意图

(2) 存储容量

存储容量指整个存储器所能存放的二进制信息量，它与磁表面大小和记录密度密切相关。

硬盘的未格式化容量是指按道密度和位密度计算出来的容量，它包括了头空、ID 域、CRC 码等信息，是可利用的所有磁化单元的总数，**未格式化容量**（或**非格式化容量**）比格式化后的实际容量要大。

对于低密度存储方式，因为每个磁道的容量相等，所以，其未格式化容量的计算方法为：

$$\text{磁盘总容量} = \text{记录面数} \times \text{理论柱面数} \times \text{内圆周长} \times \text{最内道位密度}$$

由于磁盘每面的有效记录区域是一个环，磁道在这个环内沿径向分布，所以柱面数的理论公式为：

$$\text{柱面数} = (\text{有效记录区外径} - \text{有效记录区内径}) \div 2 \times \text{道密度}$$

此外，对于每个磁道具有同样多信息的磁盘，其内圆磁道的记录密度最大，一个磁道能记录的二进制信息位的理论值应等于“内圆周长 $\times$ 最内道位密度”。

格式化后的实际容量只包含数据区。通常，**记录面数**约为盘片数的两倍。假定按每个扇区 512 字节算，则磁盘实际数据容量（也称**格式化容量**）的计算公式为：

$$\text{磁盘实际数据容量} = 2 \times \text{盘片数} \times \text{磁道数/面} \times \text{扇区数/磁道} \times 512\text{B/扇区}$$

近 30 年来扇区大小一直是 512 字节，但最近几年正在逐步更换到更大、更高效的 4096 字节扇区，通常称为**4KB 扇区**。国际硬盘设备与材料协会（IDEMA）将之称为高级格式化。

(3) 数据传输速率

数据传输速率（data transfer rate）是指磁表面存储器完成磁头定位和旋转等待以后，单位时间内从存储介质上读出或写入的二进制信息量。为区别于外部数据传输率，通常称之为**内部传输速率**（internal transfer rate），也称为**持续传输速率**（sustained transfer rate）。而**外部传输速率**（external transfer rate）是指主机中的外设控制接口从（向）外存储器的缓存读出（写入）数据的速度，由外设采用的接口类型决定。通常称外部传输速率为**突发数据传输速率**（burst data transfer rate）或**接口传输速率**。

由于磁盘在同一时刻只有一个磁头进行读写，所以内部数据传输速率等于单位时间内磁头划过的磁道弧长乘以位密度。即

$$\text{内部数据传输速率} = \text{每分钟转速} \div 60 \times \text{内圆周长} \times \text{最内道位密度}$$

(4) 平均存取时间

磁盘响应读写请求的过程如下：首先将读写请求在队列中排队，出队列后由磁盘控制器解

析请求命令，然后进行寻道、旋转等待和读写数据三个过程。

因此，总响应时间的计算公式为：

响应时间 = 排队延迟 + 控制器时间 + 寻道时间 + 旋转等待时间 + 数据传输时间

磁盘上的信息以扇区为单位进行读写，上式中后面三个时间之和称为存取时间。即

存取时间 = 寻道时间 + 旋转等待时间 + 数据传输时间

寻道时间为磁头移动到指定磁道所需时间；旋转等待时间指要读写的扇区旋转到磁头下方所需要的时间；数据传输时间（transfer time）指传输一个扇区的时间（大约 0.01ms/扇区）。由于磁头原有位置与要寻找的目的位置之间远近不一，故寻道时间和旋转等待时间只能取平均值。磁盘的平均寻道时间一般为 5 ~ 10ms，平均等待时间取磁盘旋转一周所需时间的一半，大约 4 ~ 6ms。假如磁盘转速为 6000 转/分，则平均等待时间约为 5ms。因为数据传输时间相对于寻道时间和等待时间来说非常短，所以，磁盘的平均存取时间通常近似等于平均寻道时间和平均等待时间之和。而且，磁盘第一位数据的读写延时非常长，相当于平均存取时间，而以后各位数据的读写则几乎没有延迟。

6.3.3 磁盘存储器的连接

现代计算机中，通常将复杂的磁盘物理扇区抽象成固定大小的逻辑块，物理扇区和逻辑块之间的映射由磁盘控制器来维护。磁盘控制器是一个内置固件的硬件设备，它能将主机送来的请求逻辑块号转换为磁盘的物理地址（磁头号、柱面号、扇区号），并控制磁盘驱动器进行相应的动作。

图 6.15 是磁盘驱动器（简称磁盘）通过磁盘控制器与 CPU、主存储器连接的示意图。磁盘控制器连接在 I/O 总线上，I/O 总线与其他系统总线（如处理器总线、存储器总线）之间用桥接器连接。磁盘驱动器与磁盘控制器之间的接口有多种，一般文件服务器使用 SCSI 接口，而普通的 PC 前些年多使用并行 ATA（即 IDE）接口，目前大多使用串行 ATA（即 SATA）接口。

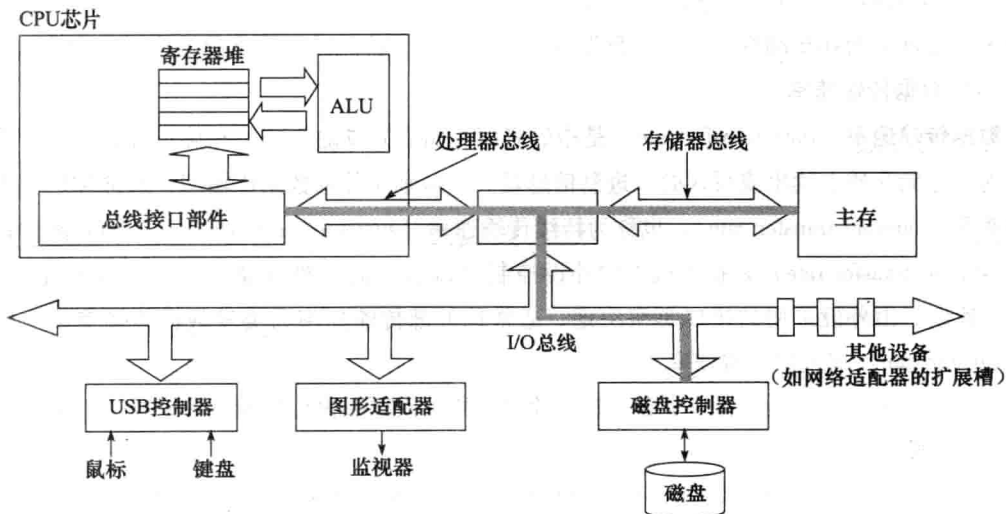


图 6.15 磁盘与 CPU、主存的连接

磁盘与主机交换数据的最小单位是扇区,因此,磁盘总是按成批数据交换方式进行读写,这种高速成批数据交换设备采用直接存储器存取 (Direct Memory Access, DMA) 方式进行数据的输入输出。该输入输出方式用专门的 DMA 接口硬件来控制外设与主存间的直接数据交换,数据不通过 CPU。通常把专门用来控制总线进行 DMA 传送的接口硬件称为DMA 控制器。在进行 DMA 传送时, CPU 让出总线控制权,由 DMA 控制器控制总线,通过“窃取”一个主存周期完成和主存之间的一次数据交换,或独占若干个主存周期完成一批数据的交换。有关 DMA 方式的实现参见 8.3.4 节。

6.3.4 固态硬盘

近年来,一种称为固态硬盘 (Solid State Disk, 简称 SSD) 的新产品开始在市场上出现 (也被称为电子硬盘)。这种硬盘并不是一种磁表面存储器,而是一种使用 NAND 闪存组成的外部存储系统,与 U 盘并没有本质差别,只是容量更大,存取性能更好。它用闪存颗粒代替了磁盘作为存储介质,利用闪存的特点,以区块写入和擦除的方式进行数据的读取和写入。电信号的控制使得固态硬盘的内部传输速率远远高于常规硬盘。有测试显示,使用固态硬盘以后,Windows 的开机速度可以提升至 20 秒以内,这是基于常规硬盘的计算机系统难以达到的速度性能。

固态硬盘刚出现时,与最高速的常规硬盘相比在读写性能方面各有上下,而且价格也较高。但随着相应技术的不断发展,目前固态硬盘的读写性能基本上超越了常规硬盘,且价格也不断下降。由于固态硬盘具有以上优点,加上其今后的发展潜力比传统硬盘要大得多,因而固态硬盘有望逐步取代传统硬盘。

固态硬盘的接口规范和定义、功能及使用方法与传统硬盘完全相同,在产品外形和尺寸上也与普通硬盘一致。目前接口标准上使用 USB、SATA 和 IDE,因此 SSD 是通过标准磁盘接口与 I/O 总线互连的。在 SSD 中有一个闪存翻译层,它将来自 CPU 的逻辑磁盘块读写请求翻译成对底层 SSD 物理设备的读写控制信号。因此,这个闪存翻译层相当于磁盘控制器。

SSD 容量通常在 32 ~ 256GB 之间,但已发表的最大容量可达 1TB。SSD 按照数据块为单位进行信息存储,每个信息块由多个页面组成,通常,页的大小为 512B 或 4KB,一个数据块由 32 ~ 128 页组成。

由于 SSD 由半导体元件构成,因此在访问性能方面比机械旋转式的磁盘要好得多。固态硬盘的顺序读和顺序写的性能基本相当,而随机读和随机写之间的性能相差 10 倍。随机读和随机写之间的性能差别是由闪存的基本属性决定的。SSD 中数据以页为单位进行读写,在写操作时,只能在要写的页所在的数据块全部被擦除后才能写,一旦某一块被擦除过,则无需再擦除。目前 SSD 平均访问时间大约是常规硬盘的 1.5 倍,写入速度可达常规硬盘的 1.5 倍,读取速度可达常规硬盘的 2 ~ 3 倍;而 CPU 占用率可比常规硬盘下降 5 ~ 10 倍。

固态硬盘目前主要的问题是使用寿命和价格。由于闪存的擦写次数有限,所以频繁擦写会降低其写入使用寿命,大约进行了 100 000 次重复写之后,数据块就会被磨损坏,而且价格上目前固态硬盘也远高于常规硬盘。但随着技术和生产工艺的不断进步,固态硬盘的写入使用寿命会不断提高,且价格也将不断下降。

6.4 高速缓冲存储器

由于 CPU 和主存所使用的半导体器件工艺不同，两者速度上的差距导致快速的 CPU 等待慢速的主存储器，为此需要想办法提高 CPU 访问主存的速度。除了提高 DRAM 芯片本身的速度和采用并行结构技术以外，加快 CPU 访存速度的主要方式之一是在 CPU 和主存之间增加高速缓冲存储器（简称高速缓存或cache）。

6.4.1 程序访问的局部性

对大量典型程序运行情况分析的结果表明，在较短时间间隔内，程序产生的地址往往集中在存储空间的一个很小范围，这种现象称为程序访问的局部性。这种局部性可细分为时间局部性和空间局部性。时间局部性是指被访问的某个存储单元在一个较短的时间间隔内很可能又被访问。空间局部性是指被访问的某个存储单元的邻近单元在一个较短的时间间隔内很可能也被访问。

出现程序访问的局部性特征的原因不难理解：程序是由指令和数据组成的，指令在主存按顺序存放，其地址连续，循环程序段或子程序段通常被重复执行，因此，指令具有明显的访问局部化特征；而数据在主存一般也是连续存放，特别是数组元素，常常被按序重复访问，因此，数据也具有明显的访问局部化特征。

例如，以下是一个 C 高级语言程序段。

```
1 sum = 0;
2 for (i = 0; i < n; i++)
3     sum += a[i];
4 *v = sum;
```

上述程序段对应的汇编程序段可由 10 条指令组成，用中间语言描述如下。

```
I0      sum ← 0
I1      ap ← A           ;A 是数组的起始地址
I2      i ← 0
I3      if (i >= n) goto done
I4 loop: t ← (ap)         ;数组元素 a[i] 的值
I5      sum ← sum + t     ;累加值在 sum 中
I6      ap ← ap + 4       ;计算下一个数组元素的地址
I7      i ← i + 1
I8      if (i < n) goto loop
I9 done: V ← sum          ;累加结果保存至地址 V
```

上述描述中的变量 *sum*、*ap*、*i*、*n*、*t* 均可认为存放在通用寄存器中，*A* 和 *V* 为主存地址。假定每条指令占 4 字节，每个数组元素占 4 字节，按字节编址，则指令和数组元素在主存中的存放情况如图 6.16 所示。

在程序执行过程中，首先，按指令 I0 ~ I3 的顺序执行，然后，

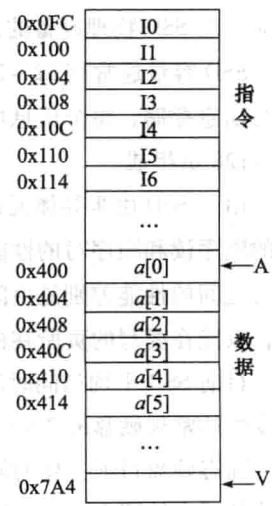


图 6.16 指令和数组在主存的存放

指令 I4 ~ I8 按顺序被循环执行 n 次。只要 n 足够大, 程序在一段时间内, 就一直在该局部区域内执行。对于取指令来说, 程序对主存的访问过程是:

0x0FC(I0) → 0x108(I3) → 0x11C(I4) → 0x11C(I8) → 0x120(I9)
↑
n 次

上述程序对数组的访问在指令 I4 中进行, 每次循环数组下标加 4, 即每次按 4 字节连续访问主存。因为数组在主存连续存放, 因此, 该程序对数据的访问过程是:

0x400 → 0x404 → 0x408 → 0x40C → …… → 0x7A4

由此可见, 在一段时间内, 访问的数据也在局部的连续区域内。

为了更好地利用程序访问的空间局部性, 通常把当前访问单元以及邻近单元作为一个主存块一起调入 cache。这个主存块的大小以及程序对数组元素的访问顺序等都对程序的性能有一定的影响。

例 6.1 假定数组元素按行优先方式存放在主存, 针对以下两段伪代码程序段 P1 和 P2, 回答下列问题。

① 对于数组 a 的访问, 哪一个空间局部性更好? 哪一个时间局部性更好?

② 变量 sum 的空间局部性和时间局部性各如何?

③ 对于指令访问来说, for 循环体的空间局部性和时间局部性如何?

程序段 P1:

```
1 int sum=array-rows(int a[M][N])
2 {
3     int i, j, sum=0;
4     for (i=0; i<M; i++)
5         for (j=0; j<N; j++)
6             sum+=a[i][j];
7     return sum;
8 }
```

程序段 P2:

```
1 int sum=array-cols(int a[M][N])
2 {
3     int i, j, sum=0;
4     for (j=0; j<N; j++)
5         for (i=0; i<M; i++)
6             sum+=a[i][j];
7     return sum;
8 }
```

解 假定 M 、 N 都为 2048, 按字节编址, 每条指令和每个数组元素各占 4 个字节, 则指令和数据在主存的存放情况如图 6.17 所示。图中 A 是数组 a 的首地址。

① 对于数组 a , 程序段 P1 和 P2 的空间局部性相差较大。程序段 P1 对数组 a 的访问顺序为 $a[0][0], a[0][1], \dots, a[0][2047], a[1][0], a[1][1], \dots, a[1][2047], \dots$ 。由此可见, 访问顺序与存放顺序是一致的, 故空间局部性好; 而程序段 P2 对数组

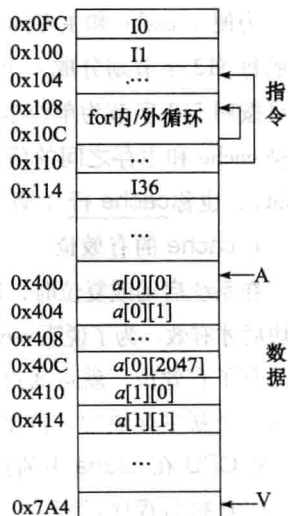


图 6.17 指令和二维数组在主存的存放

a 的访问顺序为 $a[0][0], a[1][0], \dots, a[2047][0], a[0][1], a[1][1], \dots, a[2047][1], \dots$ 。由此可见, 访问顺序与存放顺序不一致, 每次访问都要跳过 2048 个数组元素, 即 8196 个单元, 若主存与 cache 的交换单位小于 8KB, 则每次装入一个主存块到 cache 时, 下个要访问的数组元素总不能被装入 cache, 因而没有空间局部性。

数组 a 的时间局部性在程序段 P1 和 P2 中都很差, 因为每个数组元素都只被访问一次。

② 对于变量 sum , 在程序段 P1 和 P2 中的访问局部性是一样的。空间局部性对单个变量来说没有意义; 而时间局部性在 P1 和 P2 中都较好, 因为 sum 变量在 P1 和 P2 的每次循环中都要被访问。不过, 通常编译器都会将 sum 分配在寄存器中, 循环执行时只要取寄存器内容进行运算, 最后再把寄存器的值写回存储单元中, 这种情况下就无需考虑 sum 的访问局部性问题。

③ 对于 for 循环体, 程序段 P1 和 P2 中的访问局部性是一样的。因为循环体内指令按序连续存放, 所以空间局部性好; 内循环体被连续重复执行 2048×2048 次, 因此时间局部性也好。

从上述分析可以看出, 虽然程序段 P1 和 P2 的功能相同, 但因为内、外两重循环的顺序不同而导致两者对数组 a 访问的空间局部性相差较大, 从而带来执行时间的不同。曾有人将这两个程序段 ($M = N = 2048$) 放在 2GHz Pentium 4 上执行以进行比较, 其实际运行结果为: 程序段 P1 的执行只需要 59 393 288 个时钟周期, 而程序段 P2 则需要 1 277 877 876 个时钟周期。P1 比 P2 快 21.5 倍!

6.4.2 cache 的基本工作原理

cache 是一种小容量高速缓冲存储器, 由快速的 SRAM 组成, 直接制作在 CPU 芯片内, 速度较快, 几乎与 CPU 处于同一个量级。在 CPU 和主存之间设置 cache, 总是把主存中被频繁访问的活跃程序块和数据块复制到 cache 中。由于程序访问的局部性, 大多数情况下, CPU 能直接从 cache 中取得指令和数据, 而不必访问慢速的主存。

为便于 cache 和主存间交换信息, cache 和主存空间都被划分为相等的区域。例如, 将主存按照每 512 字节划分成一个区域, 同时把 cache 也划分成同样大小的区域, 这样主存中的信息就可按照 512 字节为单位送到 cache 中。我们把主存中的区域称为块 (block), 也称为主存块, 它是 cache 和主存之间的信息交换单位; cache 中存放一个主存块的区域称为行 (line) 或槽 (slot), 也称 cache 行 (槽)。

1. cache 的有效位

在系统启动或复位时, 每个 cache 行都为空, 其中的信息无效, 只有在 cache 行中装入了主存块后才有效。为了说明 cache 行中的信息是否有效, 每个 cache 行需要一个“有效位” (valid bit)。

有了有效位, 就可通过将有效位清 0 来淘汰某 cache 行中的主存块——称为冲刷 (flush), 装入一个新主存块时, 再使有效位置 1。

2. CPU 在 cache 中的访问过程

CPU 执行程序过程中, 需要从主存取指令或读写数据时, 先检查 cache 中有没有要访问的信息, 若有, 就直接在 cache 中读写, 而不用访问主存储器; 若没有, 再从主存中把当前访问信息所在的一个主存块复制到 cache 中, 因此, cache 中的内容是主存中部分内容的副本。

图 6.18 给出了带 cache 的 CPU 执行一次访存操作的过程。

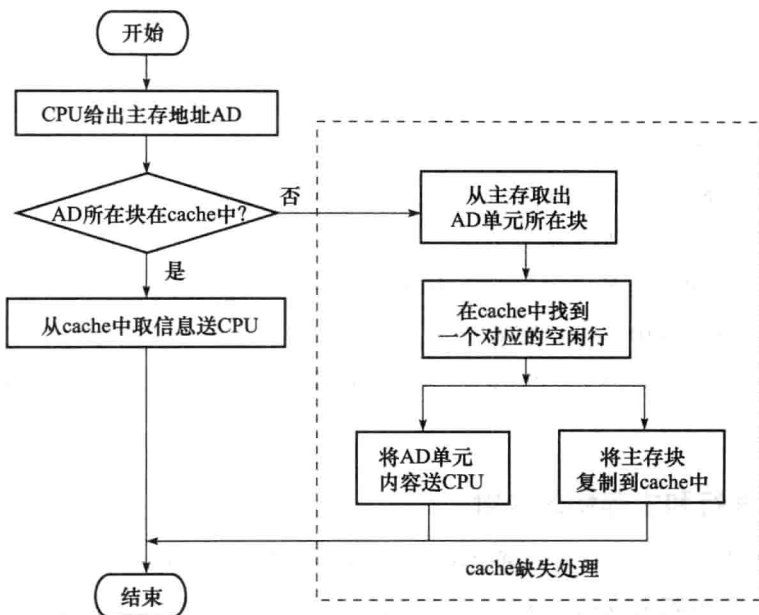


图 6.18 带 cache 的 CPU 的访存操作过程

如图 6.18 所示, 整个访存过程包括: 判断信息是否在 cache 中, 从 cache 取信息或从主存取一个主存块到 cache 等, 在对应 cache 行已满的情况下还要替换 cache 中的信息。这些工作要求在一条指令执行过程中完成, 因而只能由硬件来实现。cache 对程序员来说是透明的, 程序员编程时不用考虑信息存放在主存还是在 cache。因此, cache 位于微体系结构层面, 而不位于 ISA 层面。

3. cache-主存层次的平均访问时间

如图 6.18 所示, 在访存过程中, 需要判断所访问信息是否在 cache 中。若 CPU 访问单元所在的主存块在 cache 中, 则称cache 命中 (hit), 命中的概率称为命中率 p (hit rate), 它等于命中次数与访问总次数之比; 若不在 cache 中, 则为不命中 (miss)^①, 其概率称为缺失率 (miss rate), 它等于不命中次数与访问总次数之比。命中时, CPU 在 cache 中直接存取信息, 所用的时间开销就是 cache 访问时间 T_c , 称为命中时间 (hit time); 缺失时, 需要从主存读取一个主存块送 cache, 并同时所需信息送 CPU, 因此, 所用时间开销为主存访问时间 T_m 和 cache 访问时间 T_c 之和。通常把从主存读入一个主存块到 cache 的时间 T_m 称为缺失损失 (miss penalty)。

CPU 在 cache-主存层次的平均访问时间为:

$$T_a = p \times T_c + (1 - p) \times (T_m + T_c) = T_c + (1 - p) \times T_m$$

由于程序访问的局部性特点, cache 的命中率可以达到很高, 接近于 1。因此, 虽然缺失损

① 国内教材对“不命中”的说法有多种, 如“失效”, “失靶”、“缺失”等, 其含义一样, 本教材使用“缺失”一词。

失 >> 命中时间, 但最终的平均访问时间仍可接近 cache 的访问时间。

例 6.2 假定处理器时钟周期为 2ns, 某程序由 1000 条指令组成, 每条指令执行一次, 其中的 4 条指令在取指令时没有在 cache 中找到, 其余指令都能在 cache 中取到。在执行指令过程中, 该程序需要 3000 次主存数据访问, 其中, 6 次没有在 cache 中找到。试问:

① 执行该程序得到的 cache 命中率是多少?

② 若 cache 中存取一个信息的时间为 1 个时钟周期, 缺失损失为 10 个时钟周期, 则 CPU 在 cache-主存层次的平均访问时间为多少?

解 ① 执行该程序时的总访问次数为 $1000 + 3000 = 4000$, 未命中次数为 $4 + 6 = 10$, 故 cache 命中率为 $(4000 - 10)/4000 = 99.75\%$ 。

② 平均访问时间为 $1 + (1 - 99.75\%) \times 10 = 1.025$ 个时钟周期, 即 $1.025 \times 2\text{ns} = 2.05\text{ns}$, 与 cache 访问时间相近。 ■

6.4.3 cache 行和主存块的映射

cache 行中的信息取自主存中的某个块。在将主存块复制到 cache 行时, 主存块和 cache 行之间必须遵循一定的映射规则, 这样, CPU 要访问某个主存单元时, 可以依据映射规则到 cache 对应的行中查找要访问的信息, 而不用在整个 cache 中查找。

根据不同的映射规则, 主存块和 cache 行之间有以下三种映射方式。

① 直接 (direct): 每个主存块映射到 cache 的固定行中。

② 全相联 (full associate): 每个主存块映射到 cache 的任意行中。

③ 组相联 (set associate): 每个主存块映射到 cache 的固定组的任意行中。

以下分别介绍这三种映射方式。

1. 直接映射

直接映射的基本思想是把主存的每一块映射到固定的一个 cache 行中, 也称模映射, 其映射关系如下:

$$\text{cache 行号} = \text{主存块号} \bmod \text{cache 行数}$$

例如, 假定 cache 共有 16 行, 根据 $100 \bmod 16 = 4$, 可知主存第 100 块应映射到 cache 的第 4 行中。

直接映射方式下, 主存地址被分成标记、cache 行号和块内地址三个字段。

标记	cache 行号	块内地址
----	----------	------

假定 cache 共有 2^c 行, 主存共有 2^m 块, 主存块大小占 2^b 字节, 按字节编址, 则 cache 行号占 c 位、主存块号占 m 位, 块内地址有 b 位。因为 m 位主存块号被分解成标记字段和cache 行号字段, 因而标记字段占 $t = m - c$ 位。

图 6.19a 给出了直接映射方式下主存块和 cache 行之间的映射示意, 图中主存第 0、1、 $\dots$ 、 $2^c - 1$ 块分别映射到 cache 第 0、1、 $\dots$ 、 $2^c - 1$ 行; 主存第 2^c 、 $2^c + 1$ 、 $\dots$ 、 $2^{c+1} - 1$ 块也分别映射到 cache 第 0、1、 $\dots$ 、 $2^c - 1$ 行; 等等。每个 cache 行中还应包含一个有效位, 图 6.19 所示的 cache 行中省略了有效位。

直接映射方式下, CPU 访存过程如图 6.19b 所示。首先根据主存地址中间的 c 位, 直接找到对应的 cache 行, 将对应 cache 行中的标记和主存地址的高 t 位标记进行比较, 若相等且有效位为 1, 则访问 cache “命中”, 此时, 根据主存地址中低 b 位的块内地址, 在对应的 cache 行中存取信息; 若不相等或有效位为 0, 则 cache “缺失”, 此时, CPU 从主存中读出该主存地址所在的一块信息通过系统总线送到对应的 cache 行中, 将有效位置 1, 并将标记设置为该地址的高 t 位, 同时将该地址中的内容送 CPU。

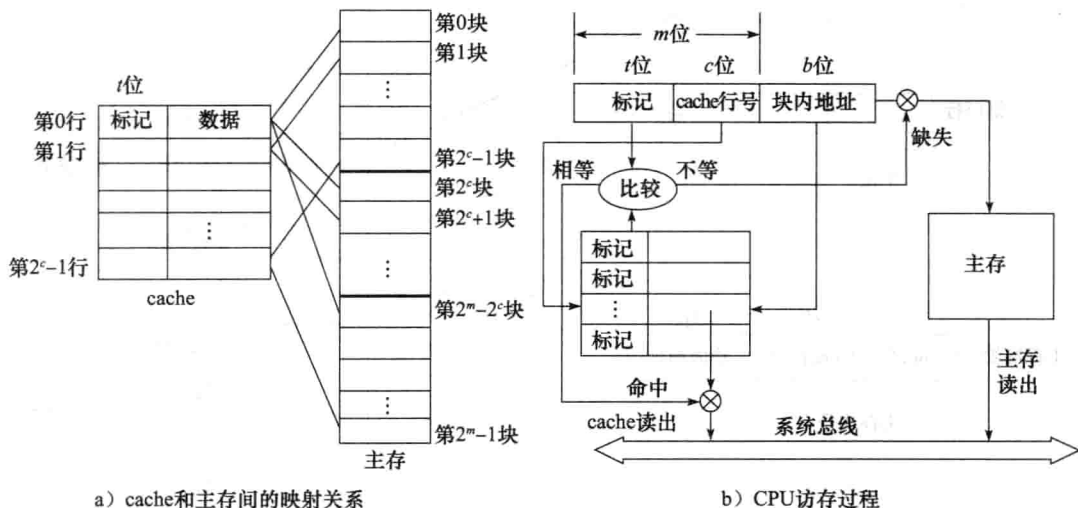


图 6.19 cache 和主存之间的直接映射方式

CPU 访存时, 读操作和写操作的过程有一些不同, 相对来说, 读操作比写操作简单。因为 cache 行中的信息是主存某块的副本, 所以, 在写操作时会出现 cache 行和主存块数据的一致性问题。下面通过例子来说明 cache 设计中的一些问题。

例 6.3 假定主存按字编址, 与 cache 之间采用直接映射方式, 块大小为 512 字。cache 数据区容量为 8K 字, 主存空间大小为 1M 字。问: 主存地址如何划分? 要求用图表示主存块和 cache 行之间的映射关系, 假定 cache 当前为空, 说明 CPU 对主存单元 0240CH 的访问过程。

解 cache 数据区容量为 8K 字 $= 2^{13}$ 字 $= 2^4$ 行 $\times 512$ 字/行 $= 16$ 行 $\times 512$ 字/行。因为主存每 16 块和 cache 的 16 行一一对应, 所以可将主存每 16 块看成一个块群, 因而, 得到主存空间地址划分为 1M 字 $= 2^{20}$ 字 $= 2^{11}$ 块 $\times 512$ 字/块 $= 2^7$ 块群 $\times 2^4$ 块/块群 $\times 2^9$ 字/块。所以, 主存地址位数 n 为 20, 标记位数 t 为 7, cache 行号位数 c 为 4, 块内地址位数 b 为 9。主存地址划分以及主存块和 cache 行的对应关系如图 6.20 所示。

主存地址 0240CH 展开为二进制数为 0000 0010 0100 0000 1100, 所以主存地址划分如下:

0000 001	0010	0 0000 1100
----------	------	-------------

根据主存地址划分可知, 该地址所在块号是 0000 001 0010 (第 18 块), 所属块群号为 0000 001 (第 1 块群), 映射到的 cache 行号为 0010 (第 2 行)。

假定 cache 开始为空, 访问 0240CH 单元的过程为: 首先根据地址中间的 4 位 0010, 找到 cache 第 2 行, 因为 cache 为空, 所以, 每个 cache 行的有效位都为 0, 因此, 不管第 2 行的标

志是否等于 0000 001，都不命中。此时，将 0240CH 单元所在的主存第 18 块复制到 cache 第 2 行，并置有效位为 1，置标记为 0000 001（表示信息取自主存的第 1 块群）。

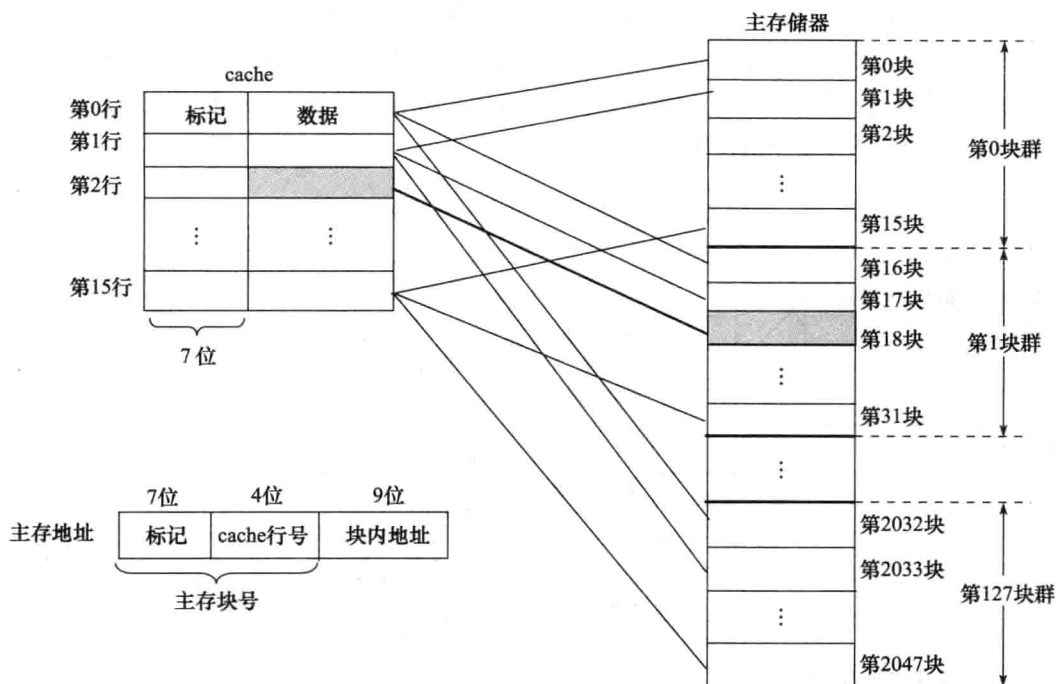


图 6.20 直接映射方式下主存块和 cache 行的对应关系

直接映射的优点是容易实现，命中时间短，但由于多个块号“同余”的内存块只能映射到同一个 cache 行，当访问集中在“同余”内存块时，就会引起频繁的调进调出，即使其他 cache 行都空闲，也毫无帮助。很显然，直接映射方式不够灵活，使得 cache 存储空间得不到充分利用，命中率较低。例如，上述例 6.3 中，若需将主存第 0 块与第 16 块同时调入 cache，由于它们都只对应 cache 第 0 行，即使其他行空闲，也总有一个主存块不能调入 cache，因此会产生频繁的调进调出。

2. 全相联映射

全相联映射的基本思想是一个主存块可装入 cache 任意一行中。全相联映射 cache 中，每行的标记用于指出该行取自主存的哪个块。因为一个主存块可能在任意一行中，所以，需要比较所有 cache 行的标记，因此，主存地址中无需 cache 行索引，只有标记和块内地址两个字段。全相联映射方式下，只要有空闲 cache 行，就不会发生冲突，因而块冲突概率低。

例 6.4 假定主存按字编址，与 cache 之间采用全相联映射，块大小为 512 字。cache 数据区容量为 8K 字，主存地址空间为 1M 字。问：主存地址如何划分？要求用图表示主存块和 cache 行之间的映射关系，并说明 CPU 对主存单元 0240CH 的访问过程。

解 cache 数据区容量为 8K 字 $= 2^{13}$ 字 $= 2^4$ 行 $\times 512$ 字/行 $= 16$ 行 $\times 2^9$ 字/行。主存地址空间为 1M 字 $= 2^{20}$ 字 $= 2^{11}$ 块 $\times 512$ 字/块。20 位的主存地址划分为两个字段：标记位数 t 为 11，块内地址位数 b 为 9。

主存地址划分以及主存块和 cache 行之间的对应关系如图 6.21 所示。

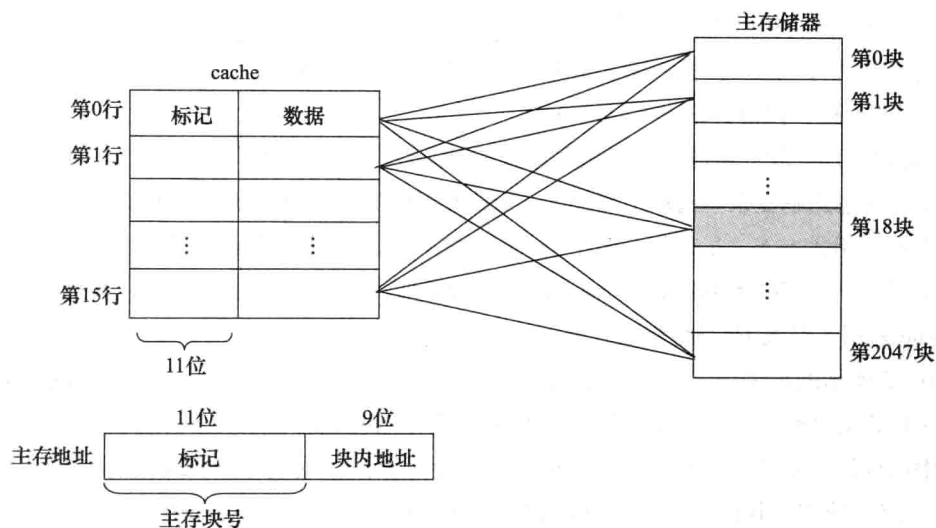


图 6.21 全相联映射方式下主存块和 cache 行的对应关系

主存地址 0240CH 展开为二进制数为 0000 0010 0100 0000 1100, 所以主存地址划分为:

0000 0010 010	0 0000 1100
---------------	-------------

访问 0240CH 单元的过程为: 首先将高 11 位标记 0000 0010 010 与 cache 中每个行的标记进行比较, 若有一个相等并且对应有效位为 1, 则命中, 此时, CPU 根据块内地址 0 0000 1100 从该行中取出信息; 若都不相等, 则不命中, 此时, 需要将 0240CH 单元所在的主存第 0000 0010 010 块 (即第 18 块) 复制到 cache 的任何一个空闲行中, 并置有效位为 1, 置标记为 0000 0010 010 (表示信息取自主存第 18 块)。

为了加快比较的速度, 通常每个 cache 行都设置一个比较器, 比较器位数等于标记字段的位数。全相联 cache 访存时根据标记字段的内容来访问 cache 行中的主存块, 它查找主存块的过程是一种“按内容访问”的存取方式, 因此, 它是一种“相联存储器”。相联映射方式的时间开销和所用元件开销都较大, 实现起来比较困难, 不适合容量较大的 cache。

3. 组相联映射

前面介绍了全相联映射和直接映射, 它们的优缺点正好相反, 两者结合可以取长补短。将两种方式结合起来产生组相联映射方式。

组相联映射的主要思想是, 将 cache 分成大小相等的组, 每个主存块被映射到 cache 固定组中的任意一行, 也即组相联采用组间模映射、组内全映射的方式。映射关系如下:

$$\text{cache 组号} = \text{主存块号} \bmod \text{cache 组数}$$

若 cache 共 16 行, 分成 8 组, 则每组有 2 行, 此时, 主存第 100 块应映射到 cache 第 4 组的任意一行中, 因为 $100 \bmod 8 = 4$ 。

组相联方式下, 主存地址被划分为标记、cache 组号和块内地址三个字段。

标记	cache 组号	块内地址
----	----------	------

假定 cache 共有 2^c 行, 被分成 2^q 组, 则每组有 $2^c/2^q = 2^{c-q}$ 行。设 $s = c - q$, 则 cache 映射方式称为 2^s 路组相联映射, 即 $s = 1$ 为 2 路组相联; $s = 2$ 为 4 路组相联, 以此类推。若主存共有 2^m 块, 主存块大小占 2^b 字节, 按字节编址, 则块内地址有 b 位, cache 组号有 q 位, 标记和 cache 组号共 m 位, 因而标记占 $t = m - q$ 位。

s 的选取决定了块冲突的概率和相联比较的复杂性。 s 越大, 则 cache 发生块冲突的概率越低, 相联比较电路越复杂。选取适当的 s , 可使组相联映射的成本比全相联的低得多, 而性能上仍可接近全相联方式。早几年, 由于 cache 容量不大, 所以通常 $s = 1$ 或 2, 即 2 路或 4 路组相联较常用, 但随着技术的发展, cache 容量不断增加, s 的值有增大的趋势, 目前有许多处理器的 cache 采用 8 路或 16 路组相联方式。

图 6.22 所示的是采用 2 路组相联映射的 cache, 其中每个 cache 行都有对应的有效位 V 、标记 Tag 和数据 Data。整个访存过程为: ① 根据主存地址中的 cache 组号找到对应组; ② 将主存地址中的标记与对应组中每个行的标记 Tag 进行比较; ③ 将比较结果和有效位 V 相“与”; ④ 若有一路比较相等并有效位为 1, 则输出“Hit”(命中)为 1, 并选中这一路 cache 行中的主存块; ⑤ 在“Hit”为 1 的情况下, 根据主存地址中的块内地址从选中的一块内取出对应单元的信息, 若“Hit”不为 1, 则 CPU 要到主存去读一块信息到 cache 行中。

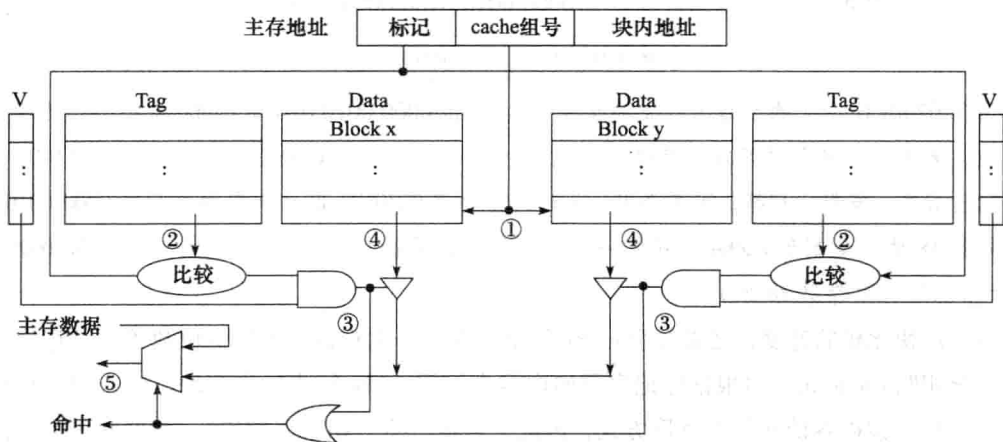


图 6.22 组相联映射方式的硬件实现

例 6.5 假定主存按字编址, 与 cache 之间采用 2 路组相联映射, 块大小为 512 字。cache 数据区容量为 8K 字, 主存地址空间为 1M 字。问: 主存地址如何划分? 要求用图表示主存块和 cache 行之间的映射关系, 并说明 CPU 对主存单元 01202H 的访问过程。

解 cache 数据容量为 8K 字 $= 2^{13}$ 字 $= 2^3$ 组 $\times 2^1$ 行/组 $\times 512$ 字/行。主存地址空间为 1M 字 $= 2^{20}$ 字 $= 2^{11}$ 块 $\times 512$ 字/块 $= 2^8$ 组群 $\times 2^3$ 块/组群 $\times 2^9$ 字/块。因此, 主存地址位数 n 为 20, 标记位数 t 为 8, 组号位数 q 为 3, 块内地址位数 b 为 9。

主存地址划分以及主存块和 cache 行的对应关系如图 6.23 所示。

主存地址 01202H 展开为二进制数是 0000 0001 0010 0000 0010, 所以主存地址划分为:

0000 0001	001	0 0000 0010
-----------	-----	-------------

访问 01202H 单元的过程为：根据地址中间 3 位 001，找到 cache 第 1 组，将标记 0000 0001 与第 1 组中两个 cache 行的标记同时进行比较。若有一个相等并且有效位是 1，则命中，此时，根据低 9 位块内地址从对应行中取出单元内容送 CPU；若都不相等或有一个相等但有效位为 0，则不命中，此时，将 01202H 单元所在的主存第 0000 0001 001 块（即第 9 块）复制到 cache 第 001 组（即第 1 组）的任意一个空闲行中，并置有效位为 1，置标记为 0000 0001（表示信息取自主存第 1 组群）。

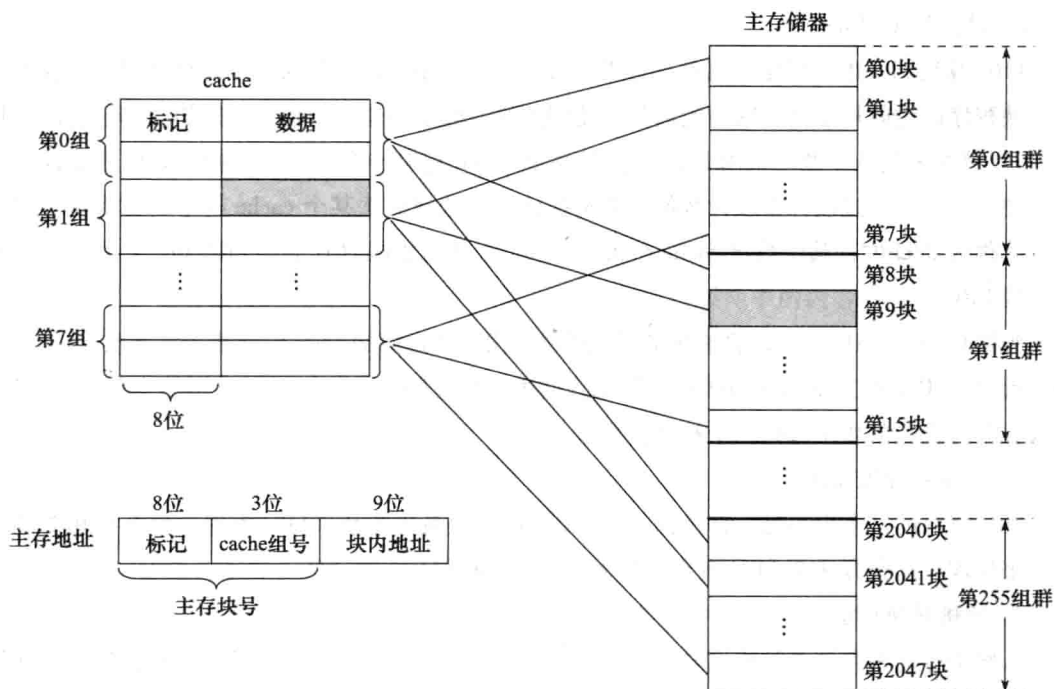


图 6.23 组相联映射方式下主存块和 cache 行的对应关系

组相联映射方式结合了直接映射和全相联映射的优点。当 cache 的组数为 1 时，变为全相联映射；当每组只有一个 cache 行时，则变为直接映射。组相联映射的冲突概率比直接映射低，由于只有组内各行采用全相联映射，所以比较器的位数和个数都比全相联映射少，易于实现，查找速度也快得多。

6.4.4 cache 中主存块的替换算法

cache 行数比主存块数少得多，因此，往往多个主存块会映射到同一个 cache 行中。当新的一个主存块复制到 cache 时，cache 中的对应行可能已经全部被占满，此时，必须选择淘汰掉一个 cache 行中的主存块。例如，对于例 6.5 中的 2 路组相联映射 cache，假定第 0 组的两个行分别被主存第 0 块和第 8 块占满，此时若需调入主存第 16 块，根据映射关系，它只能存放到 cache 第 0 组，因此，已经在第 0 组的主存第 0 块和第 8 块这两块中，必须选择调出其中一块，到底调出哪一块呢？这就是淘汰策略问题，也称为替换算法或替换策略。

常用的替换算法有：先进先出（First-In-First-Out，简称 FIFO）、最近最少用（Least-Re-

cently Used, 简称 LRU)、最不经常用 (Least-Frequently Used, 简称 LFU) 和随机替换等。可以根据实现的难易程度以及是否能获得较高的命中率这两方面来决定采用哪种算法。

1. 先进先出算法

FIFO 算法的基本思想是: 总是选择最早装入 cache 的主存块被替换掉。这种算法实现起来较方便, 但不能正确反映程序的访问局部性, 因为最先进入的主存块也可能是目前经常要用的, 因此, 这种算法有可能产生较大的缺失率。

2. 最近最少用算法

LRU 算法的基本思想是: 总是选择近期最少使用的主存块被替换掉。这种算法能比较正确地反映程序的访问局部性, 因为当前最少使用的块一般来说也是将来最少被访问的。它的实现比 FIFO 算法要复杂一些。采用 LRU 算法的每个 cache 行有一个计数器, 用计数值来记录主存块的使用情况, 通过硬件修改计数值, 并根据计数值选择淘汰某个 cache 行中的主存块。这个计数值称为 LRU 位, 其位数与 cache 组大小有关。2 路组相联时有 1 位 LRU 位, 4 路组相联时有 2 位 LRU 位。

为简化上述 LRU 位计数的硬件实现, 通常采用一种近似的 LRU 位计数方式来实现 LRU 算法。近似 LRU 计数方法仅区分哪些是新调入的主存块, 哪些是较长时间未用的主存块, 然后, 在较长时间未用的块中选择一个被替换出去。

3. 最不经常用算法

LFU 算法的基本思想是: 替换掉 cache 中引用次数最少的块。LFU 也用与每个行相关的计数器来实现。这种算法与 LRU 有点类似, 但不完全相同。

4. 随机替换算法

从候选行的主存块中随机选取一个淘汰掉, 与使用情况无关。模拟试验表明, 随机替换算法在性能上只稍逊于基于使用情况的算法, 而且代价低。

6.4.5 cache 一致性问题

因为 cache 中的内容是某些主存块的副本, 当 CPU 进行写操作需对 cache 中的内容进行更新时, 就存在 cache 和主存如何保持一致的问题。除此之外, 以下情况也会出现 cache 一致性问题。

① 当多个设备都允许访问主存时。例如, 像磁盘这类高速 I/O 设备可通过 DMA 方式直接读写主存, 如果 cache 中的内容被 CPU 修改而主存块没有更新的话, 则从主存传送到 I/O 设备的内容就无效; 若 I/O 设备修改了主存块的内容, 则对应 cache 行中的内容就无效。

② 当多个 CPU 都带有各自的 cache 而共享主存时。在多 CPU 系统中, 若某个 CPU 修改了自身 cache 中的内容, 则对应的主存块和其他 CPU 中对应的 cache 行的内容都变为无效。

解决 cache 一致性问题的关键是处理好写操作。通常有两种写操作方式。

1. 全写法

全写法 (write through) 的基本做法是: 当 CPU 执行写操作时, 若写命中, 则同时写 cache 和主存; 若写不命中, 则有以下两种处理方式。

① 写分配法 (write allocate)。先在主存块中更新相应存储单元, 然后分配一个 cache 行, 将更新后的主存块装入分配的 cache 行中。这种方式可以充分利用空间局部性, 但每次写不命中都要从主存读一个块到 cache 中, 增加了读主存块的开销。

② 非写分配法 (not write allocate)。仅更新主存单元而不把主存块装入 cache 中。这种方式可以减少读入主存块的时间, 但没有很好利用空间局部性。

由此可见, 该方式实际上采用的是对主存块信息及其所有副本信息全都直接同步更新的做法, 因此通常被称为通写法或直写法, 也有教材称之为写直达法。

显然, 全写法在替换时不必将被替换的 cache 内容写回主存, 而且 cache 和主存的一致性能得到充分保证。但是, 这种方法会大大增加写操作的开销。例如, 假定一次写主存需要 100 个 CPU 时钟周期, 那么 10% 的存储 (store) 指令就使得 CPI 增加了 $100 \times 10\% = 10$ 个时钟周期。

为了减少写主存的开销, 通常在 cache 和主存之间加一个写缓冲 (write buffer)。在 CPU 写 cache 的同时, 也将信息写入写缓冲, 然后由存储控制器将写缓冲中的内容写入主存。写缓冲是一个 FIFO 队列, 一般只有几项, 在写操作频率不是很高的情况下, 因为 CPU 只需将信息写入快速的写缓冲而无需写慢速的主存, 因而效果较好。但是, 如果写操作频繁发生, 则会使写缓冲饱和而发生阻塞。

2. 回写法

回写法 (write back) 的基本做法是: 当 CPU 执行写操作时, 若写命中, 则信息只被写入 cache 而不被写入主存; 若写不命中, 则在 cache 中分配一行, 将主存块调入该 cache 行中并更新 cache 中相应单元的内容。因此, 该方式下在写不命中时, 通常采用写分配法进行写操作。

在 CPU 执行写操作时, 回写法不会更新主存单元, 只有当 cache 行中的主存块被替换时, 才将该主存块内容一次性写回主存。这种方式的好处在于减少了写主存的次数, 因而大大降低了主存带宽需求。为了减少写回主存块的开销, 每个 cache 行设置了一个修改位 (dirty bit, 有时也称为“脏位”)。若修改位为 1, 则说明对应 cache 行中的主存块被修改过, 替换时需要写回主存; 若修改位为 0, 则说明对应主存块未被修改过, 替换时无需写回主存。

由此可见, 该方式实际上采用的是回头再写或最后一次性写的做法, 因此通常被称为回写法或一次性写方式, 也有教材称之为写回法。

由于回写法没有同步更新 cache 和主存内容, 所以存在 cache 和主存内容不一致而带来的潜在隐患。通常需要其他的同步机制来保证存储信息的一致性。

6.4.6 影响 cache 性能的因素

决定系统访存性能的重要因素之一是 cache 命中率, 它与许多因素有关。命中率与关联度有关, 关联度越高, 命中率越高。关联度反映一个主存块对应的 cache 行的个数, 显然, 直接映射的关联度为 1; 2 路组相联映射的关联度为 2, 4 路组相联映射的关联度为 4, 全相联映射的关联度为 cache 行数。同时, 命中率与 cache 容量也有关。显然, cache 容量越大, 命中率就

越高。此外，命中率还与主存块的大小有一定关系。采用大的交换单位能很好地利用空间局部性，但是，较大的主存块需要花费较多的时间来存取，因此，缺失损失会变大。由此可见，主存块的大小必须适中，不能太大，也不能太小。

除了上述提到的这些因素外，设计 cache 时还要考虑以下因素：采用单级还是多级 cache、数据 cache 和指令 cache 是分开还是合在一起、主存—总线—cache—CPU 之间采用什么架构等，甚至主存 DRAM 芯片的内部结构、存储器总线的总线事务类型等，也都与 cache 设计有关，都会影响系统总体性能。下面对这些问题进行简单分析说明。

1. 单级/多级 cache、联合/分离 cache 的选择问题

早期采用的是单级片外 cache，近年来，多级片内 cache 系统已成为主流。目前 cache 基本上都在 CPU 芯片内，且使用 L1 和 L2 cache，甚至有 L3 cache，CPU 的访问顺序为 L1 cache、L2 cache 和 L3 cache。通常 L1 cache 采用分离 cache，即数据 cache 和指令 cache 分开设置，分别存放数据和指令。指令 cache 有时称为代码 cache (code cache)。L2 cache 和 L3 cache 为联合 cache，即数据和指令放在一个 cache 中。

由于多级 cache 中各级 cache 所处的位置不同，使得对它们的设计目标有所不同。例如，假定是两级 cache。那么，对于 L1 cache，通常更关注速度而不要求有很高的命中率，因为，即使不命中，还可以到 L2 cache 中访问，L2 cache 的速度比主存速度快得多；而对于 L2 cache，则要求尽量提高其命中率，因为若不命中，则必须到慢速的主存中访问，其缺失损失会很大而影响总体性能。

2. 主存—总线—cache 间的连接结构问题

在主存和 cache 之间传输的单位是主存块，要使缺失损失最小，必须在主存、总线和 cache 之间构建快速的传输通道。什么样的连接结构才能使主存块在主存和 cache 之间的传输速度最快呢？

为了计算将主存块传送到 cache 所用的时间，必须先了解 CPU 从主存取一块信息到 cache 的过程。从主存读一块数据到 cache，一般包含以下三个阶段。

- 发送地址和读命令到主存：假定用 1 个时钟周期；
- 主存准备好一个数据：假定用 10 个时钟周期；
- 从总线传送一个数据：假定用 1 个时钟周期。

主存、总线和 cache 之间可以有三种连接方式：① 窄型结构，即在主存、总线和 cache 之间每次按一个字的宽度进行传送；② 宽型结构，即在它们之间每次传送多个字；③ 交叉存储结构，即主存采用多模块交叉存取方式，在主存、总线和 cache 之间每次按一个字的宽度进行传送。假定一个主存块有 4 个字，那么对于这三种结构，其缺失损失各是多少呢？

图 6.24 给出了三种方式下的主存块传送过程。图 6.24a 对应于窄型结构，连续进行“送地址—读—传送”4 次，每次一个字，其缺失损失为 $4 \times (1 + 10 + 1) = 48$ 个时钟周期。图 6.24b 对应于宽度为两个字的宽型结构，连续进行“送地址—读—传送”两次，每次两个字，其缺失损失为 $2 \times (1 + 10 + 1) = 24$ 个时钟周期；假定宽型结构的宽度为 4 个字，则只要进行“送地址—读—传送”一次，其缺失损失为 $1 \times (1 + 10 + 1) = 12$ 个时钟周期。图 6.24c 对

应于交叉存储结构,在首地址送出后,每隔一个时钟启动一个存储模块,第1个模块用10个时钟周期准备好第1个字,然后在总线上传送第1个字,同时,第2个模块准备好第2个字,总线上传输第2个字的同时,第3个模块准备好第3个字,总线上传输第3个字的同时,第4个模块准备好第4个字,最后总线传送第4个字,因此,其缺失损失为 $1 + 1 \times 10 + 4 \times 1 = 15$ 个时钟周期。通过以上分析可看出,交叉存储结构的性价比最好。

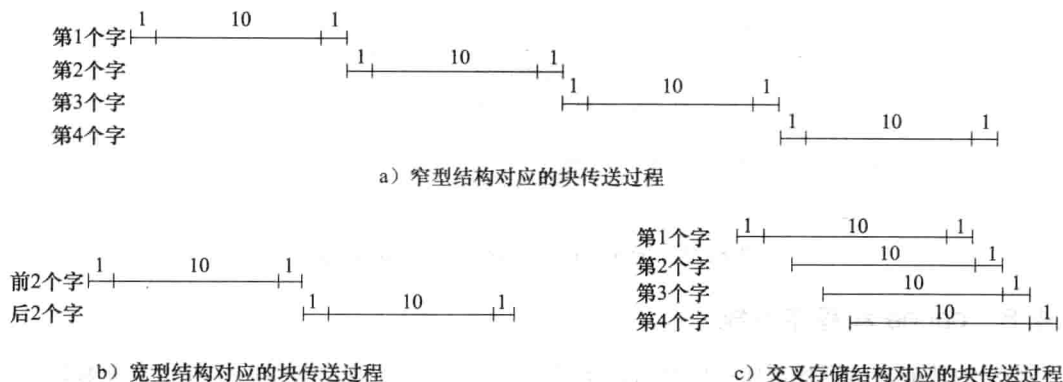


图 6.24 主存块在主存—总线—cache 之间的传送过程

6.4.7 IA-32 的 cache 结构举例

现代计算机系统中几乎都使用 cache 机制,以下以 Intel 公司微处理器中的 cache 为例来说明具体的 cache 结构。

Pentium 微处理器在芯片内集成了一个代码 cache 和一个数据 cache。片内 cache 采用两路组相联结构,共 128 组,每组两行。片内 cache 采用 LRU 替换策略,每组有一个 LRU 位,用来表示该组哪一路中的 cache 行被替换。Pentium 处理器有两条单独的指令来清除或回写 cache。Pentium 处理器采用片外二级 cache,可配置为 256KB 或 512KB,也采用两路组相联方式,每行数据有 32、64 或 128 字节。

Pentium 4 处理器芯片内集成了一个 L2 cache 和两个 L1 cache。L2 cache 是联合 cache,数据和指令存放在一起,所有从主存获取的指令和数据都先送到 L2 cache 中。它有三个端口,一个对外,两个对内。对外的端口通过预取控制逻辑和总线接口部件,与处理器总线相连,用来和主存交换信息。对内的端口中,一个以 256 位位宽与 L1 数据 cache 相连;另一个以 64 位位宽与指令预取部件相连,由指令预取部件取出指令,送指令译码器,指令译码器再将指令转换为微操作序列,送到指令 cache 中。Intel 称该指令 cache 为踪迹高速缓存 (Trace Cache, 简称 TC),其中存放的并不是指令;而是指令对应的微操作序列。

Intel Core i7 采用的 cache 结构如图 6.25 所示,每个核内有各自私有的 L1 cache 和 L2 cache。其中,L1 指令 cache 和数据 cache 都是 32KB 数据区,皆为 8 路组相联,存取时间都是 4 个时钟周期;L2 cache 是联合 cache,共有 256KB 数据区,8 路组相联,存取时间是 11 个时钟周期。该多核处理器中还有一个供所有核共享的 L3 cache,其数据区大小为 8MB,16 路组相联,存取时间是 30~40 个时钟周期。Intel Core i7 中所有 cache 的块大小都是 64B。

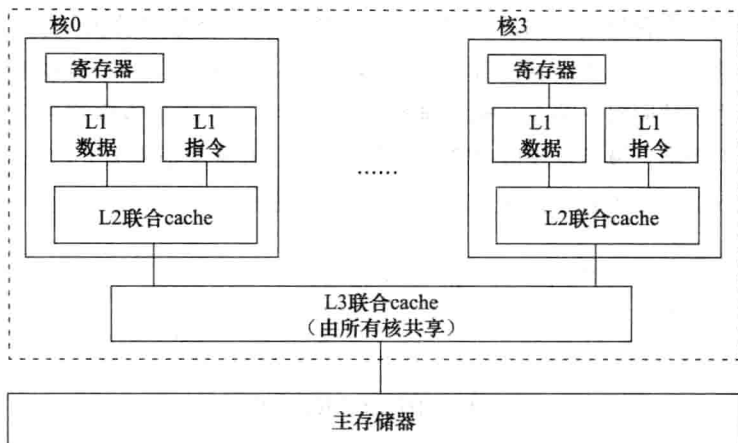


图 6.25 Intel Core i7 处理器的 cache 结构

6.4.8 cache 和程序性能

程序的性能指执行程序所用的时间，显然，程序的性能与程序执行时访问指令和数据所用的时间有很大关系，而指令和数据的访问时间与相应的 cache 命中率、命中时间和缺失损失有关。对于给定的计算机系统而言，命中时间和缺失损失是确定的，因此，指令和数据的访存时间主要由 cache 命中率决定，而 cache 命中率则主要由程序的空间局部性和时间局部性决定。因此，为了提高程序的性能，程序员须编写出具有良好访问局部性的程序。

考虑程序的访问局部性通常是在数据的访问局部性上下工夫，而数据的访问局部性又主要是指数组、结构等类型数据访问时的局部性，这些数据结构的数据元素访问通常是通过循环语句进行的，所以，如何合理地处理循环，特别是内循环，对于数据访问局部性来说是非常重要的。下面通过几个例子来说明不同的循环处理将带来不同的程序性能。

例 6.6 某计算机的主存地址空间大小为 256MB，按字节编址。指令 cache 和数据 cache 分离，均有 8 个 cache 行，主存与 cache 交换的块大小为 64B，数据 cache 采用直接映射方式。现有两个功能相同的程序 A 和 B，其伪代码如图 6.26 所示。

程序A:

```
int a[256][256];
.....
int sum_array1( )
{
    int i, j, sum = 0;
    for ( i = 0; i < 256; i++)
        for ( j = 0; j < 256; j++)
            sum += a[i][j];
    return sum;
}
```

程序B:

```
int a[256][256];
.....
int sum_array2( )
{
    int i, j, sum = 0;
    for ( j = 0; j < 256; j++)
        for ( i = 0; i < 256; i++)
            sum += a[i][j];
    return sum;
}
```

图 6.26 例 6.6 的伪代码程序

假定 int 类型数据用 32 位补码表示, 程序编译时 i 、 j 、 sum 均分配在寄存器中, 数组 a 按行优先方式存放, 其首地址为 320(十进制数)。请回答下列问题, 要求说明理由或给出计算过程。

① 若不考虑用于 cache 一致性维护和替换算法的控制位, 则数据 cache 的总容量为多少?

② 数组元素 $a[0][31]$ 和 $a[1][1]$ 各自所在主存块对应的 cache 行号分别是多少 (行号从 0 开始)?

③ 程序 A 和 B 的数据访问命中率各是多少? 哪个程序的执行时间更短?

解 ① cache 中的每一行信息除了用于存放主存块的数据区外, 还有有效位、标记信息以及用于 cache 一致性维护的修改位 (dirty bit) 和用于替换算法的使用位 (如 LRU 位) 等控制位。因为主存地址空间大小为 256MB, 因而主存地址为 28 位, 其中 6 位为块内地址, 3 位为 cache 行号 (行索引), 标志信息有 $28 - 6 - 3 = 19$ 位。因此, 在不考虑用于 cache 一致性维护和替换算法的控制位的情况下, 数据 cache 的总容量为 $8 \times (19 + 1 + 64 \times 8) = 4256$ 位 = 532 字节。

② 对于某个数组元素所在主存块对应的 cache 行号的计算方法有以下三种。

方法一: 要得到某个数组元素所在块对应的 cache 行号, 最简单的做法就是把该数组元素的地址计算出来, 然后根据地址求出主存块号, 最后用主存块号除以 8 取余数 (即主存块号 mod 8) 就是对应的 cache 行号。因为每个数组元素为一个 32 位 int 型变量, 故占 4 个字节。因此, $a[0][31]$ 的地址为 $320 + 4 \times 31 = 444$, $[444/64] = 6$ (取整), 也即 $a[0][31]$ 对应的主存块号为 6。因为 $6 \bmod 8 = 6$, 所以对应的 cache 行号为 6。

方法二: 将地址转换为 28 位二进制数, 然后取出其中的行索引 (即行号) 字段的值, 得到对应行号。也即, 将地址 444 转换为二进制表示为 0000 0000 0000 0000 000 110 111100, 中间 3 位 110 为行号 (行索引), 因此, 对应的 cache 行号为 6。

方法三: 用画图的方式也可以清楚地表示 cache 行和主存块之间的映射关系。

同理, 数组元素 $a[1][1]$ 对应的 cache 行号为 $[(320 + 4 \times (1 \times 256 + 1))/64] \bmod 8 = 5$ 。

③ 编译时 i 、 j 、 sum 均分配在寄存器中, 故数据访问命中率仅需要考虑数组 a 的访问情况。

- 程序 A 的数据访问命中率。计算程序 A 的数据访问命中率可采用以下两种方法:

方法一: 由于程序 A 中数组访问顺序与存放顺序相同, 故依次访问的数组元素位于相邻单元; 程序共访问 256×256 次 = 64K 次, 占 $64K \times 4B/64B = 4K$ 个主存块; 因为首地址正好位于一个主存块的边界, 故每次将一个主存块装入 cache 时, 总是第一个数组元素缺失, 其他都命中, 共缺失 4K 次。因此, 数据访问的命中率为 $(64K - 4K)/64K = 93.75\%$ 。

方法二: 因为每个主存块的命中情况都一样, 所以也可以按每个主存块的命中率计算。主存块大小为 64B, 包含有 16 个数组元素, 因此, 共访存 16 次, 其中第一次不命中, 因而命中率为 $15/16 = 93.75\%$ 。

- 程序 B 的数据访问命中率。由于程序 B 中的数组访问顺序与存放顺序不同, 依次访问的数组元素分布在相隔 $256 \times 4 = 1024$ 的单元处, 它们都不在同一个主存块中; 因为数据 cache 只有 8 行, 而每次内循环要调入 $256 \times 4/64B = 16$ 个主存块, 因此, 以前被装入到 cache 的主存块, 当需要再次访问其中的数组元素时, 已经被替换出 cache, 因而不能命中。由此可知, 所有访问都不能命中, 命中率为 0。

程序 A 的命中率高, 因此, 程序 A 的执行时间比程序 B 的执行时间短。 ■

例 6.7 通过对方格中每个点设置相应的 CMYK 值就可以将方格涂上相应的颜色。

图 6.27 中的三个程序段都可实现对一个 8×8 的方格中涂上黄颜色的功能。

<pre> struct pt_color { int c; int m; int y; int k; } struct pt_color sq[8][8]; int i, j; for (i=0; i<8; i++) { for (j=0; j<8; j++) { sq[i][j].c = 0; sq[i][j].m = 0; sq[i][j].y = 1; sq[i][j].k = 0; } } </pre>	<pre> struct pt_color { int c; int m; int y; int k; } struct pt_color sq[8][8]; int i, j; for (i=0; i<8; i++) { for (j=0; j<8; j++) { sq[j][i].c = 0; sq[j][i].m = 0; sq[j][i].y = 1; sq[j][i].k = 0; } } </pre>	<pre> struct pt_color { int c; int m; int y; int k; } struct pt_color sq[8][8]; int i, j; for (i=0; i<8; i++) for (j=0; j<8; j++) sq[i][j].y = 1; for (i=0; i<8; i++) for (j=0; j<8; j++) { sq[i][j].c = 0; sq[i][j].m = 0; sq[i][j].k = 0; } </pre>
a) 程序段A	b) 程序段B	c) 程序段C

图 6.27 例 6.7 中的伪代码程序

假设 cache 数据区大小为 512B, 采用直接映射方式, 块大小为 32B, 存储器按字节编址, $\text{sizeof}(\text{int}) = 4$ 。编译时变量 i 和 j 分配在寄存器中, 数组 sq 按行优先方式存放在 00000C80H 开始的连续区域中, 主存地址为 32 位。要求:

- ① 对三个程序段 A、B、C 中数组访问的时间局部性和空间局部性进行分析比较。
- ② 画出主存中的数组元素和 cache 行的对应关系图。
- ③ 计算三个程序段 A、B、C 中数组访问的写操作次数、写不命中次数和写缺失率。

解 ① 程序段 A、B 和 C 中, 都是每个数组元素只被访问一次, 所以都没有时间局部性。程序段 A 中数组元素的访问顺序和存放顺序一致, 所以, 空间局部性好; 程序段 B 中数组元素的访问顺序和存放顺序不一致, 所以, 空间局部性不好; 程序段 C 中数组元素的访问顺序和存放顺序部分一致, 所以空间局部性的优劣介于程序 A 和 B 之间。

② cache 行数为 $512\text{B}/32\text{B} = 16$ 。数组首地址为 00000C80H, 因为 00000C80H 正好是主存第 1100100B(100) 块的起始地址, 所以数组从主存第 100 块开始存放。一个数组元素占 $4 \times 4\text{B} = 16\text{B}$, 所以每 2 个数组元素占用一个主存块。 8×8 的数组共占用 32 个主存块, 正好是 cache 数据区大小的 2 倍。因为 $100 \bmod 16 = 4$, 所以主存第 100 块映射的 cache 行号为 4。主存中的数组元素与 cache 行的映射关系如图 6.28 所示。

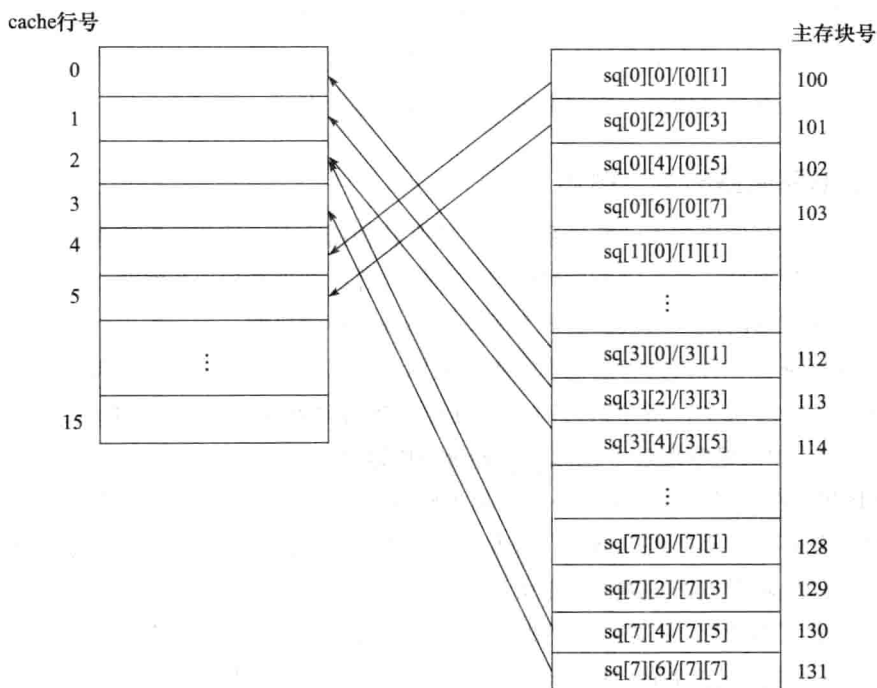


图 6.28 主存中数组元素与 cache 行的映射关系

③ 对于程序段 A：每两个数组元素（共涉及 8 次写操作）装入一个 cache 行中，总是第一次访问时未命中，后面 7 次都命中，所以，总的写操作次数为 $64 \times 4 = 256$ 次，写不命中次数为 $256 \times 1/8 = 32$ 次，因而写缺失率为 12.5%。

对于程序段 B：每两个数组元素（共涉及 8 次写操作）装入一个 cache 行中，总是只有一个数组元素（涉及 4 次写操作）在淘汰之前被访问，并且总是第一次不命中，后面 3 次命中。即写不命中次数为 $256 \times 1/4 = 64$ 次，因而写缺失率为 25%。

对于程序段 C：第一个循环共访问 64 次，每次装入两个数组元素，第一次不命中，第二次命中；第二个循环共访问 64×3 次，每两个数组元素（共涉及 6 次写操作）装入一个 cache 行中，并且总是第一次不命中，后面 5 次命中。所以总的写不命中次数为 $32 + (3 \times 64) \times 1/6 = 64$ 次，因而总的写缺失率为 25%。

6.5 虚拟存储器

目前计算机主存主要由 DRAM 芯片构成，由于技术和成本等原因，主存的存储容量受到限制，并且各种不同计算机所配置的物理内存容量多半也不相同，而程序设计时人们显然不希望受到特定计算机的物理内存大小的制约，因此，如何解决这两者之间的矛盾是一个重要问题；此外，现代操作系统都支持多道程序运行，如何让多个程序有效而安全地共享主存是另一个重要问题。

为了解决上述两个问题，计算机中采用了虚拟存储技术。其基本思想是，程序员在一个不

受物理内存空间限制并且比物理内存空间大得多的虚拟的逻辑地址空间中编写程序，就好像每个程序都独立拥有一个巨大的存储空间一样。程序执行过程中，把当前执行到的一部分程序和相应的数据调入主存，其他暂不用的部分暂时存放在磁盘上。

6.5.1 虚拟存储器的基本概念

在不采用虚拟存储机制的计算机系统中，CPU 执行指令时，取指令和存取操作数所用的地址都是主存的物理地址，无需进行地址转换，因而计算机硬件结构比较简单，指令执行速度较快。实时性要求较高的嵌入式微控制器大多不采用虚拟存储机制。

目前，在服务器、台式机和笔记本等各类通用计算机系统中都采用虚拟存储技术。在采用虚拟存储技术的计算机中，指令执行时，通过存储器管理部件（Memory Management Unit，简称 MMU）将指令中的逻辑地址（也称虚拟地址或虚地址，简称为 VA）转化为主存的物理地址（也称主存地址或实地址，简称为 PA）。在地址转换过程中由硬件检查是否发生了访问信息不在主存或地址越界或访问越权等情况。若发现信息不在主存，则由操作系统将数据从磁盘读到主存。若发生地址越界或访问越权，则由操作系统进行相应的异常处理。由此可以看出，虚拟存储技术既解决了编程空间受限的问题，又解决了多道程序共享主存带来的安全等问题。图 6.29 是具有虚拟存储机制的 CPU 与主存的连接示意图，从图中可知，CPU 执行指令时所给出的是指令或操作数的虚拟地址，需要通过 MMU 将虚拟地址转换为主存的物理地址才能访问主存，MMU 包含在 CPU 芯片中。图中显示 MMU 将一个虚拟地址转换为物理地址 4，从而将第 4、5、6、7 这 4 个单元的数据组成 4 字节数据送到 CPU。图 6.29 仅是一个简单示意图，其中没有考虑 cache 等情况。

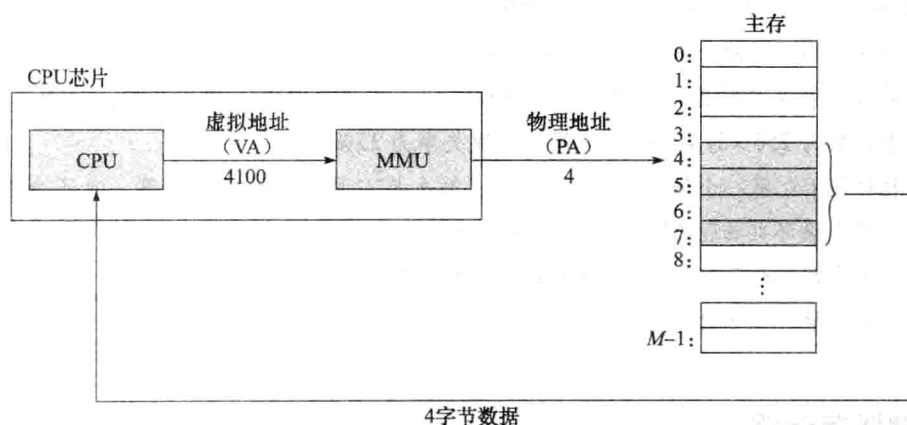


图 6.29 具有虚拟存储机制的 CPU 和主存的连接

虚拟存储机制（简称虚存机制）由硬件与操作系统共同协作实现，涉及计算机系统许多层面，包括操作系统中的许多概念，如进程、存储器管理、虚拟地址空间、缺页处理等。

进程是一个具有一定独立功能的程序关于某个数据集合的一次运行活动，简单来说，进程就是程序的一次执行过程。每一个进程都有它自己的地址空间，一般情况下，地址空间包括只读区（代码和只读数据）、可读可写数据区（初始化数据和未初始化数据）、动态的堆区和栈

区。一个静态的程序只有装入系统运行后，它才成为一个活动的实体，才能称为进程。因为进程是与一个用户程序（即应用程序）对应的概念，因此，很多时候也称其为用户进程。

6.5.2 虚拟地址空间

在第4章4.5节中提到，每个高级语言源程序经编译、汇编、链接等处理生成可执行的二进制机器目标代码时，都被映射到同样的虚拟地址空间（参见图4.20），因此，所有进程的虚拟地址空间是一致的，这简化了链接器的设计和实现，也简化了程序的加载过程。

虚拟存储机制为程序提供了一个极大的虚拟地址空间（也称为逻辑地址空间），它是主存和磁盘存储器的抽象。虚存机制带来了一个假象，使得每个进程好像都独占使用主存，并且主存空间极大。这有三个好处：① 每个进程具有一致的虚拟地址空间，从而可以简化存储管理；② 它把主存看成是磁盘存储器的一个缓存，在主存中仅保存当前活动的程序段和数据区，并根据需要在磁盘和主存之间进行信息交换，通过这种方式，使有限的主存空间得到了有效利用；③ 每个进程的虚拟地址空间是私有的，因此，可以保护各自进程不被其他进程破坏。

例如，图6.30给出了在IA-32+Linux操作系统下hello程序的一个进程对应的虚拟地址空间映像。

虚拟地址空间分为两大部分：内核虚拟存储空间和用户虚拟存储空间，分别简称为内核空间（kernel space）和用户空间（user space）。

内核空间在0xc0000000以上的高端地址上，用来存放操作系统内核代码和数据等，其中内核代码和数据区在每个进程的地址空间中都相同。用户程序没有权限访问内核区。

用户空间用来存放进程的代码和数据等，它又被分为以下几个区域。

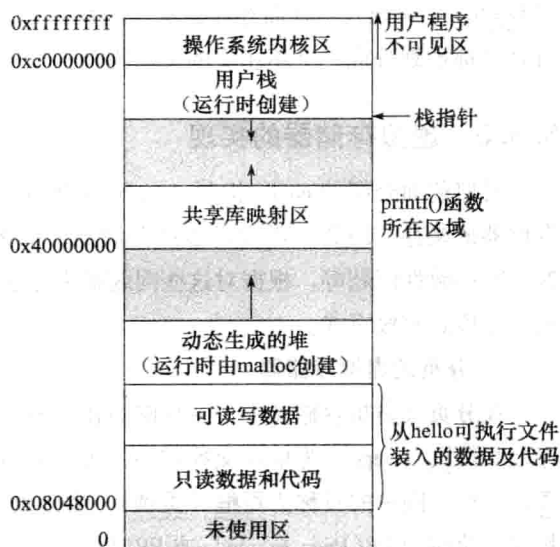


图 6.30 Linux 虚拟地址空间

① 用户栈（user stack）。用来存放程序运行时过程调用的参数、返回值、返回地址、过程局部变量等，随着程序的执行，该区会不断地从高地址向低地址增长或向反方向减退。

② 共享库（shared library）。用来存放公共的共享函数库代码，如hello中的printf()函数等。

③ 堆（heap）。用于动态申请存储区，例如，C语言中用malloc()函数分配的存储区，或C++中用new操作符分配的存储区。申请一块内存时，动态地从低地址向高地址增长，可用free()函数或delete操作符释放相应的一块内存区。

④ 可读写数据区。存放进程中的静态全局变量，堆区从该区域的结尾处开始向高地址增长。

⑤ 只读数据和代码区。存放进程中的代码和只读数据，如hello进程中的程序代码和字符

串“hello, world\n”。

每个区域都有相应的起始位置，堆区和栈区相向生长，栈区从内核起始位置 0xc0000000 开始向低地址增长，栈区和堆区合起来称为堆栈，其中的共享库映射区从 0x40000000 开始向高地址增长。只读区（代码和只读数据）从 0x08048000 开始向高地址增长。这些存储区域与可执行目标文件中不同的节之间的对应关系请参见第 4 章图 4.20。

为了便于对存储空间的管理和存储保护，在规划存储映像时，通常将内核空间 and 用户空间分在两端。在用户空间中又把动态区域和静态区域分在两端，动态区域中把过程调用时的动态局部信息（栈区）和动态分配的内存区（堆区）分在两端，静态区域中把可读写区和只读区分在两端。这样的存储映像，便于每个区域的访问权限设置，因而有利于存储保护和存储管理。

从图 6.30 可以看出，一个进程的虚拟地址空间中有一些“空洞”。例如，堆区和栈区都是动态生长的，因而在栈和共享库映射区之间、堆和共享库映射区之间都可能没有内容存在，这些没有和任何内容相关联的页称为“未分配页”；对于代码和数据等有内容的区域所关联的页面，称为“已分配页”。已分配页中又有两类：已调入主存而被缓存在 DRAM 中的页面称为“缓存页”；未调入主存而存在磁盘上的页称为“未缓存页”。因此，任何时刻一个进程中的所有页面都被划分成三个不相交的页面集合：未分配页集合、缓存页集合和未缓存页集合。

6.5.3 虚拟存储器的实现

对照前面介绍的 cache 机制（cache 是主存的缓存），可以把 DRAM 构成的主存看成是磁盘存储器的缓存。因此，要实现虚拟存储器，也必须考虑交换块大小问题、映射问题、替换问题、写一致性问题等。根据对这些问题解决方法的不同，虚拟存储器分成三种不同类型：分页式、分段式和段页式。

1. 分页式虚拟存储器

在分页式虚拟存储器中，主存储器和虚拟地址空间都被划分成大小相等的页面，磁盘和主存之间按页（page）为单位交换信息。通常把虚拟地址空间中的页称为虚拟页、逻辑页或虚页；主存空间中的页称为页框（页帧）、物理页或实页。有时虚拟页简称为 VP（Virtual Page），物理页简称为 PF（Page Frame）或 PP（Physical Page）。对于这些概念的名称，不同教材的说法可能不同，但含义是一样的。

虚拟存储器管理方式采用“请求分页”思想，每次访问仅将当前需要的页面调入主存，而进程中其他不活跃的页面放在磁盘上。当访问某个信息所在页不在主存时发生缺页异常，此时，硬件将调出 OS 内核中的缺页处理程序，将缺失页面从磁盘调入主存。

与主存块大小相比，虚拟页的大小要大得多。因为 DRAM 比 SRAM 大约慢 10~100 倍，而磁盘比 DRAM 大约慢 100 000 多倍，所以进行缺页处理所花的代价要比 cache 缺失损失大得多。而且，根据磁盘的特性，磁盘扇区定位所用的时间要比磁盘读写一个数据的时间长大约 100 000 倍，也即对扇区第一个数据的读写比随后数据的读写要慢 100 000 倍。考虑到缺页代价的巨大和磁盘访问第一个数据的开销，通常将主存和磁盘之间交换的页的大小设定得比较大，典型的有 4KB 和 8KB 等，而且有越来越大的趋势。

因为缺页处理代价较大，所以提高命中率是关键，因此，在主存页框和虚拟页之间采用全

相联映射方式。此外，当进行写操作时，由于磁盘访问速度很慢，所以，不能每次写操作都同时写 DRAM 和磁盘，因而，在处理一致性问题时，采用回写（write back）方式，而不用全写（write through）方式。

因为在虚拟存储机制中采用全相联映射，所以每个虚拟页可以存放到对应主存区域的任何一个空闲页框中。因此，与 cache 一样，必须要有一种方法来确定虚拟地址空间中各个虚拟页所存放的主存页框号或者在磁盘上的存储位置。虚拟存储管理机制中用页表（page table）来描述这种对应关系。

（1）页表

页表除了可以描述虚拟页的存放位置以外，还可以对每个虚拟页的访问权限、使用情况、修改情况等进行说明。操作系统在主存中给每个进程都生成了一个页表，进程中的每个虚拟页在页表中都有一个对应的表项，称为页表项。页表项内容包括该虚拟页的存放位置、装入位（valid）、修改位（dirty）、使用位、访问权限位和禁止缓存位等。

页表项中的存放位置字段用来建立虚拟页和物理页框之间的映射，用于进行虚拟地址到物理地址的转换。装入位也称为有效位或存在位，用来表示对应页面是否在主存，若为“1”，表示该虚拟页已从外存调入主存，是一个“缓存页”，此时，存放位置字段指向主存物理页号（即页框号或实页号）；若为“0”，则表示没有被调入主存，此时，若存放位置字段为 null，则说明是一个“未分配页”，否则是一个“未缓存页”，其存放位置字段给出该虚拟页在磁盘上的起始地址。修改位（也称脏位）用来说明页面是否被修改过，虚存机制中采用回写策略，利用修改位可判断替换时是否需写回磁盘。使用位用来说明页面的使用情况，配合替换策略来设置，因此也称替换控制位，例如，是否最先调入（FIFO 位），是否最近最少用（LRU 位）等。访问权限位用来说明页面是可读可写、只读还是只可执行等，用于存储保护。禁止缓存位用来说明页面是否可以装入 cache，通过正确设置该位，可以保证磁盘、主存和 cache 数据的一致性。

图 6.31 给出了一个页表的示例，其中有 4 个缓存页：VP1、VP2、VP5 和 VP7；有两个未分配页：VP0 和 VP4；有两个未缓存页：VP3 和 VP6。

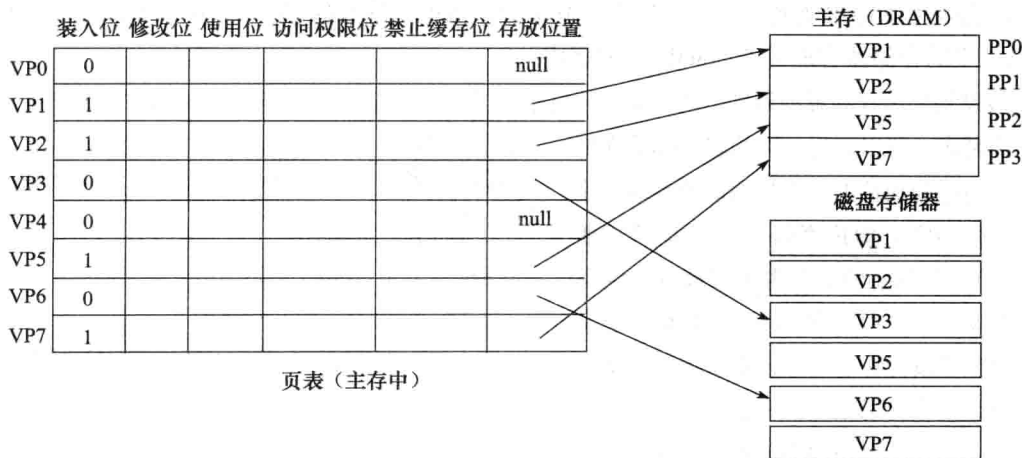


图 6.31 主存中的页表示例

对于图 6.31 所示的页表，假如 CPU 执行一条指令要求访问某个数据，若该数据正好在虚拟页 VP1 中，则根据页表得知，VP1 对应的装入位为 1，该页的信息存放在物理页 PP0 中，因此，可通过地址转换部件将虚拟地址转换为物理地址，然后到 PP0 中访问该数据；若该数据在 VP6 中，则根据页表得知，VP6 对应的装入位为 0，表示页面缺失，发生缺页异常，需要调出操作系统的缺页异常处理程序进行处理。缺页异常处理程序根据页表中 VP6 对应表项的存放位置字段，从磁盘中将所缺失的页面读出，然后找一个空闲的物理页框存放该页信息。若主存中没有空闲的页框，则还要选择一个页面淘汰出来替换到磁盘上。因为采用回写策略，所以页面淘汰时，需根据修改位确定是否要写回磁盘。缺页处理过程中需要对页表进行相应的更新，缺页异常处理结束后，程序回到原来发生缺页的指令继续执行。

对于图 6.31 所示的页表，虚拟页 VP0 和 VP4 是未分配页，但随着进程的动态执行，可能会使这些未分配页中有了具体的数据。例如，调用 malloc 函数会使堆区增长，若新增的堆区正好与 VP4 对应，则操作系统内核就在磁盘上分配一个存储空间给 VP4，用于存放新增堆区中的内容，同时，对应 VP4 的页表项中的存放位置字段被填上该磁盘空间的起始地址，VP4 从未分配页转变为未缓存页。

系统中每个进程都有一个页表，页表属于进程控制信息，存放在虚拟地址空间的内核空间，页表在主存的首地址记录在页表基址寄存器中。页表的项数由虚拟地址空间大小决定。前面提到，虚拟地址空间是一个用户编程不受其限制的足够大的地址空间。因此，页表项数会很多，因而会带来页表过大的问题。例如，在 Intel x86 系统中，虚拟地址为 32 位，页面大小为 4KB，因此，一个进程有 $2^{32}/2^{12} = 2^{20}$ 个页面，也即每个进程的页表可达 2^{20} 个页表项。若每个页表项占 32 位，则一个页表的大小为 4MB。显然，这么大的页表全部放在主存中是不适合的。

解决页表过大的方法有很多，可以采用限制大小的一级页表或者二级页表、多级页表方式，也可以采用哈希方式的倒置页表等方案。如何实现主要是操作系统考虑的问题，在此不多赘述。

(2) 地址转换

对于采用虚存机制的系统，指令中给出的地址是虚拟地址，CPU 执行指令时，首先要将虚拟地址转换为主存物理地址，才能到主存取指令和数据。地址转换 (address translation) 工作由 CPU 中的存储器管理部件 (MMU) 来完成。

假设每个进程的虚拟地址空间有 m 页，主存中有 n 个页框 (通常情况下 $m \geq n$)。由于页大小是 2 的幂次，所以，每一页的起点都落在低位字段为零的地址上。虚拟地址分为两个字段：高位字段为虚拟页号 (即虚页号或逻辑页号)，低位字段为页内偏移地址 (简称页内地址)。主存物理地址也分为两个字段：高位字段为物理页号，低位字段为页内偏移地址。由于虚拟页和物理页的大小一样，所以两者的页内偏移地址是相等的。

页式虚拟存储管理方式下，地址变换过程如图 6.32 所示。

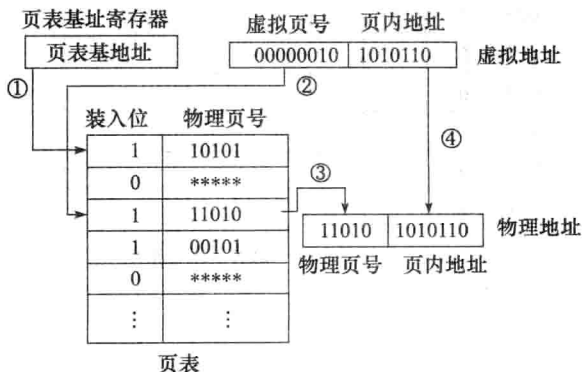


图 6.32 页式虚存的地址转换

转换过程如下：首先根据页表基址寄存器的内容，找到主存中对应的页表起始位置，然后以虚拟地址高位字段的虚拟页号作为索引，找到对应的页表项。若装入位为1，则取出物理页号，并与虚拟地址中的页内地址拼接，形成访问主存时实际的物理地址；若装入位为0，则说明缺页，需要操作系统进行缺页处理。

(3) 快表

从上述地址转换过程可看出，访存时首先要到主存查页表，然后才能根据转换得到的物理地址再访问主存以存取指令或数据。如果缺页，则还要进行页面替换、页表修改等，访问主存的次数就更多。因此，采用虚拟存储机制后，使得访存次数增加了。为了减少访存次数，往往把页表中最活跃的几个页表项复制到高速缓存中，这种在高速缓存中的页表项组成的页表称为后备转换缓冲器（Translation Lookaside Buffer，简称 TLB），通常称为快表，相应地称主存中的页表为慢表。

这样，在地址转换时，首先到快表中查页表项，如果命中，则无需访问主存中的页表。因此，快表是减少访存时间开销的有效方法。

快表比页表小得多，为提高命中率，快表通常具有较高的关联度，大多采用全相联或组相联方式。每个表项的内容由页表项内容加上一个 TLB 标记字段组成，TLB 标记字段用来表示该表项取自页表中哪个虚拟页对应的页表项。因此，TLB 标记字段的内容在全相联方式下就是该页表项对应的虚拟页号；组相联方式下则是对应虚拟页号的高位部分，而虚拟页号的低位部分作为TLB 组索引用于选择 TLB 组。

图 6.33 是一个具有 TLB 和 cache 的多级层次化存储系统示意图，图中 TLB 和 cache 都采用组相联映射方式。

在图 6.33 中，CPU 给出的是一个 32 位的虚拟地址，首先，由 CPU 中的 MMU 进行虚拟地址到物理地址的转换；然后，由处理 cache 的硬件根据物理地址进行存储访问。

MMU 对 TLB 查表时，20 位的虚拟页号被分成标记（Tag）和组索引两部分，首先由组索引确定在 TLB 的哪一组进行查找。查找时将虚拟页号的标记部分与 TLB 中该组每个标记字段同时进行比较，若有某个相等且对应有效位 V 为 1，则 TLB 命中，此时，可直接通过 TLB 进行地址转换；否则 TLB 缺失，此时，需要访问主存去查慢表。图中所示的是两级页表方式，虚拟页号被分成项目索引和页表索引两部分，根据这两部分可得到对应的页表项，从而进行地址转换，并将对应页表项的内容送入 TLB 形成一个新的 TLB 表项，同时，将虚拟页号的高位部分作为 TLB 标记填入新的 TLB 表项中。若 TLB 已满，还要进行 TLB 替换，为降低替换算法开销，TLB 常采用随机替换策略。

在 MMU 完成地址转换后，cache 硬件根据映射方式将转换得到的主存物理地址划分成多个字段，然后，根据 cache 索引，找到对应的 cache 行或 cache 组，将对应各 cache 行中的标记与物理地址中的高位地址进行比较，若相等且对应有效位为 1，则 cache 命中，此时，根据块内地址取出对应的字，需要的话，再根据字节偏移量从字中取出相应字节送 CPU。

目前 TLB 的一些典型指标为：TLB 大小为 16 ~ 512 项，块大小为 1 ~ 2 项（每个表项 4 ~ 8B），命中时间为 0.5 ~ 1 个时钟周期，缺失损失为 10 ~ 100 个时钟周期，命中率为 90% ~ 99%。

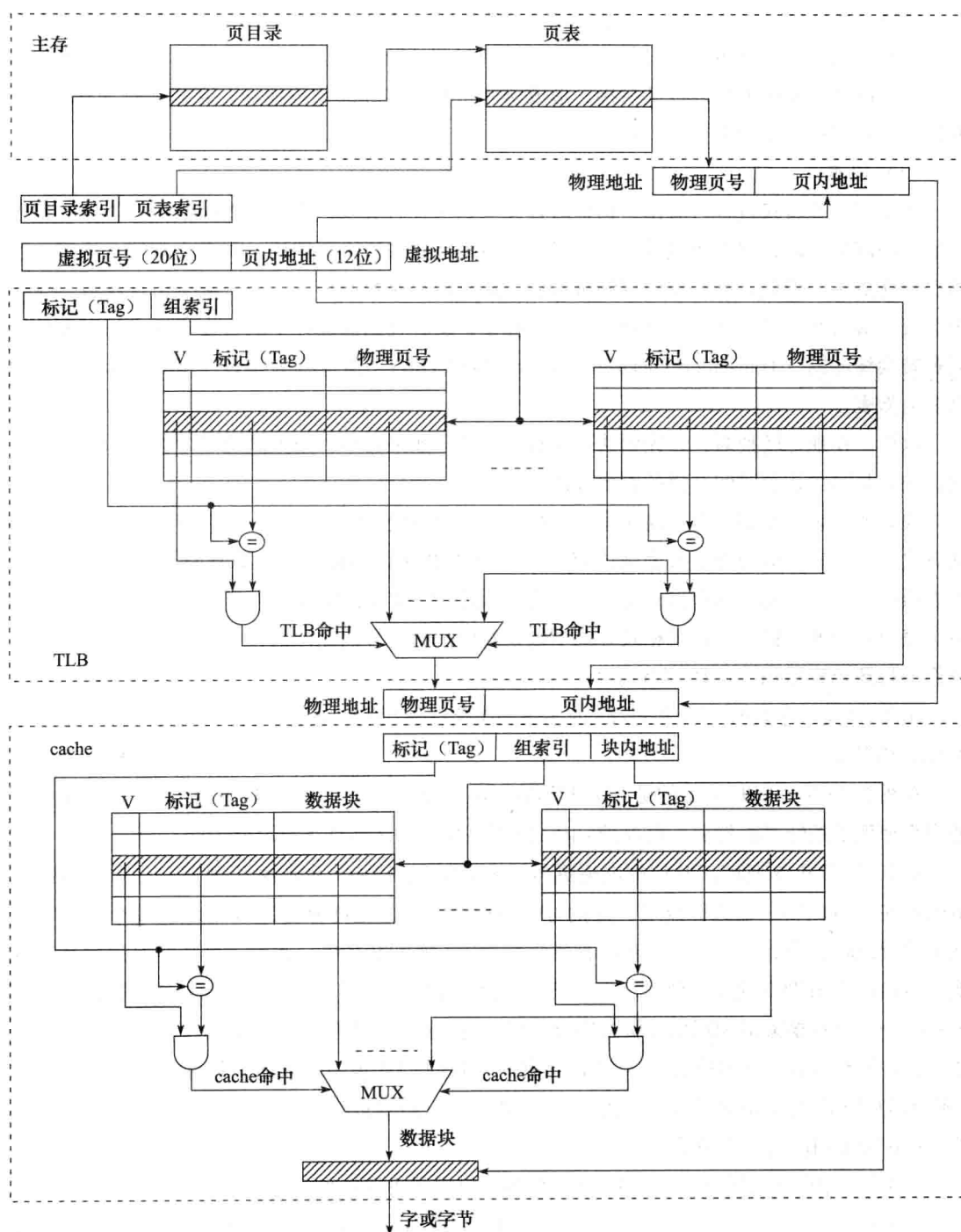


图 6.33 TLB 和 cache 的访问过程

(4) CPU 访存过程

在一个具有 cache 和虚拟存储器的系统中, CPU 的一次访存操作可能涉及 TLB、页表、cache、主存和磁盘的访问, 其访问过程如图 6.34 所示。

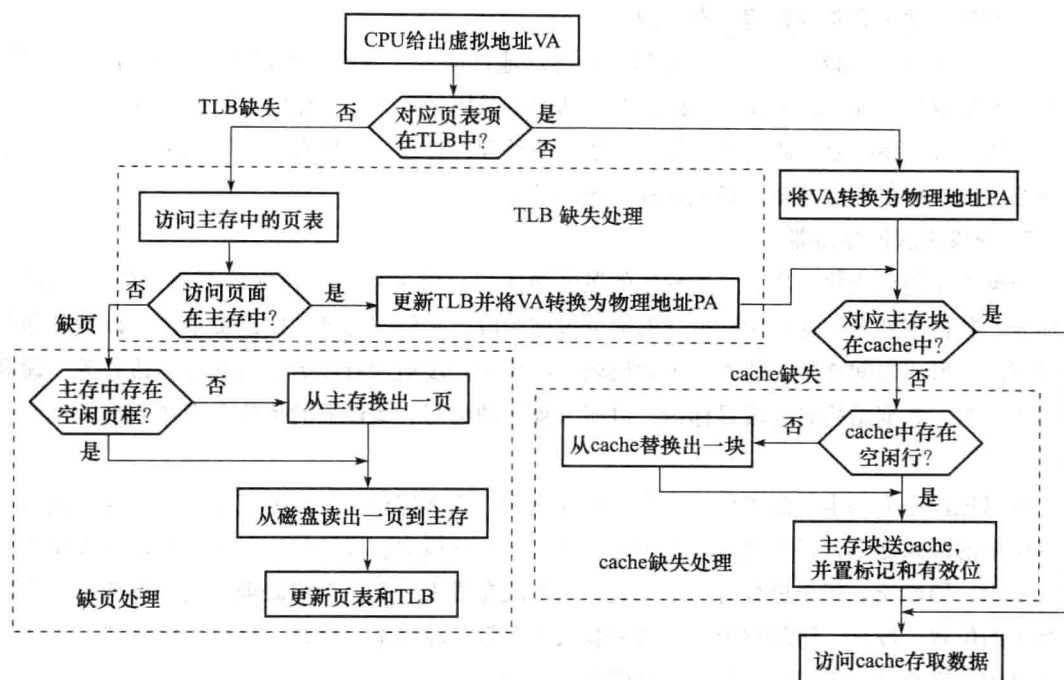


图 6.34 CPU 访存过程

从图 6.34 可以看出, CPU 访存过程中存在以下三种缺失情况。

- ① TLB 缺失 (TLB miss): 要访问的虚拟页对应的页表项不在 TLB 中。
- ② cache 缺失 (cache miss): 要访问的主存块不在 cache 中。
- ③ 缺页 (page miss): 要访问的虚拟页不在主存中。

表 6.1 给出了三种缺失的几种组合情况。

表 6.1 TLB、page、cache 三种缺失组合

序号	TLB	page	cache	说 明
1	hit	hit	hit	可能, TLB 命中则页一定命中, 信息在主存, 就可能在 cache 中
2	hit	hit	miss	可能, TLB 命中则页一定命中, 信息在主存, 但可能不在 cache 中
3	miss	hit	hit	可能, TLB 缺失但页可能命中, 信息在主存, 就可能在 cache 中
4	miss	hit	miss	可能, TLB 缺失但页可能命中, 信息在主存, 但可能不在 cache 中
5	miss	miss	miss	可能, TLB 缺失, 则页也可能缺失, 信息不在主存, 一定也不在 cache
6	hit	miss	miss	不可能, 页缺失, 说明信息不在主存, TLB 中一定没有该页表项
7	hit	miss	hit	不可能, 页缺失, 说明信息不在主存, TLB 中一定没有该页表项
8	miss	miss	hit	不可能, 页缺失, 说明信息不在主存, cache 中一定也没有该信息

很显然, 最好的情况是第 1 种组合, 此时, 无需访问主存; 第 2、3 两种组合, 都需要访问一次主存; 第 4 种组合要访问两次主存; 第 5 种组合会发生“缺页”异常, 需访问磁盘, 并至少访问主存两次。

cache 缺失处理由硬件完成; 缺页处理由软件完成, 操作系统通过缺页异常处理程序来实现; 而对于 TLB 缺失, 则既可以用硬件也可以用软件来处理。用软件方式处理时, 操作系统通

过专门的TLB 缺失异常处理程序来实现。

对于页式虚拟存储器，其页面的起点和终点地址固定。因此，实现简单，开销少。但是，由于页不是逻辑上独立的实体，因此，对于那些不采用对齐方式存储的计算机来说，可能会出现一个数据或一条指令分跨在不同页等问题，使处理、管理、保护和共享等都不方便。采用下面介绍的段式虚拟存储器就可避免这种情况的发生。

2. 分段式虚拟存储器

根据程序的模块化性质，可按程序的逻辑结构划分成多个相对独立的部分，例如，过程、数据表、数据阵列等。这些相对独立的部分被称为段，它们作为独立的逻辑单位可以被其他程序段调用，形成段间连接，从而产生规模较大的程序。段通常有段名、段起点、段长等。段名可用用户名、数据结构名或段号标识，以便于程序的编写、编译器的优化和操作系统的调度管理等。

可以把段作为基本信息单位在主存—辅存之间传送和定位。分段方式下，将主存空间按实际程序中的段来划分，每个段在主存中的位置记录在段表中，段的长度可变，所以段表中需有长度指示，即段长。每个进程有一个段表，每个段在段表中有一个段表项，用来指明对应段在主存中的位置、段长、访问权限、使用 and 装入情况等。段表本身也是一个可再定位段，可以存在外存中，需要时调入主存，但一般驻留在主存中。

在分段式虚拟存储器中，虚拟地址由段号和段内地址组成。通过段表把虚拟地址转换成主存物理地址，其转换过程如图 6.35 所示。

每个进程的段表在内存的首地址存放在段表基址寄存器中，根据虚拟地址中的段号，可找到对应段表项，以检查是否存在以下三种异常情况。

- ① 缺段（段不存在）：装入位 = 0。
- ② 地址越界：偏移量超出最大段长。
- ③ 访问越权：操作方式与指定访问权限

不符。

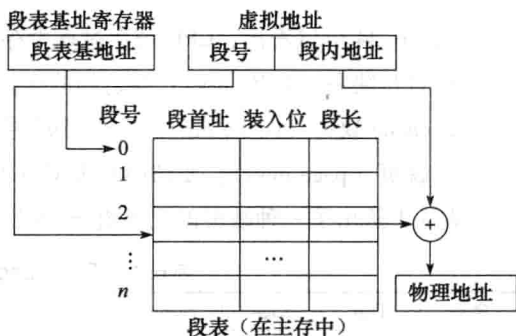


图 6.35 分段式虚存的地址转换

若发生以上三种情况，则调用相应的异常处理程序，否则，将段表项中的段首址与虚拟地址中的段内地址相加，生成访问主存时的物理地址。

因为段本身是程序的逻辑结构所决定的一些独立部分，因而分段对程序员（实际上是编译器）来说是不透明的；而分页方式则对编译器透明，即编译器不需知道程序如何分页。

分段式管理系统的优点是段的分界与程序的自然分界相对应；段的逻辑独立性使它易于编译、管理、修改和保护，也便于多道程序共享；某些类型的段（如堆、栈、队列等）具有动态可变长度，允许自由调度以便有效利用主存空间。但是，由于段的长度各不相同，段的起点和终点不定，给主存空间分配带来麻烦，而且容易在主存中留下许多空白的零碎空间，造成浪费。

分段式和分页式存储管理各有优缺点，因此可采用两者相结合的段页式存储管理方式。

3. 段页式虚拟存储器

在段页式虚拟存储器中，程序按模块分段，段内再分页，用段表和页表（每段一个页表）

进行两级定位管理。段表中每个表项对应一个段，每个段表项中包含一个指向该段页表起始位置的指针，以及该段其他的控制和存储保护信息，由页表指明该段各页在主存中的位置以及是否装入、修改等状态信息。

程序的调入调出按页进行，但它又可以按段实现共享和保护。因此，它兼有分页式和分段式存储管理的优点。它的缺点是在地址映象过程中需要多次查表。

6.5.4 存储保护

为避免主存中多道程序相互干扰，防止某进程出错而破坏其他进程的正确性，或某进程不合法地访问其他进程的代码或数据区，应该对每个进程进行存储保护。

为了对操作系统的存储保护提供支持，硬件必须具有以下三种基本功能。

① 使部分 CPU 状态只能由操作系统内核程序写，而进程只能读不能写。

例如，对于页表首地址、TLB 内容等，只有操作系统内核程序才能用特殊指令（一般称为管态指令或特权指令）来写。常用的特权指令有刷新 cache、刷新 TLB、改变特权模式、停止处理器执行等。

② 支持至少两种特权模式。

操作系统内核程序比用户程序具有更多的特权，例如，内核程序可以执行用户程序不能执行的特权指令，内核程序可以访问用户程序不能访问的存储空间等，为了区分这种特权，需要为内核程序和用户程序设置不同的特权级别或运行模式。

执行内核程序时处理器所处的模式称为管理模式（supervisor mode）、内核模式（kernel mode）、超级用户模式或管理程序状态，简称管态、管理态、内核态或者核心态；执行用户程序时处理器所处的模式称为用户模式（user mode）、用户状态或目标程序状态，简称为目态或用户态。本教材中将分别使用内核态和用户态表示两个特权模式。

需要说明的是，这里的特权模式与 IA-32 处理器的工作模式不是一回事，但是两者之间具有非常密切的关系。IA-32 工作在实地址模式下不区分特权级，只有在保护模式下才区分特权级。IA-32 支持 4 个特权级，但操作系统通常只使用第 0 级（内核态）和第 3 级（用户态）。

③ 提供让 CPU 在内核态和用户态之间相互转换的机制。

如果用户进程需要访问内核代码和数据，那么必须通过系统调用接口（执行陷阱指令）来间接访问。响应异常和中断可使 CPU 从用户态转到内核态。异常和中断处理后的返回指令（return from exception）可使 CPU 从内核态转到用户态。有关异常和中断的详细内容在第 7 章和第 8 章介绍。

硬件通过提供相应的专用寄存器、专门的指令、专门的状态/控制位等，与操作系统一起实现上述三个功能。通过这些功能，并把页表保存在操作系统的地址空间中，操作系统就可更新页表，并防止用户进程改变页表，以确保用户进程只能访问由 OS 分配给它的存储空间。

存储保护包括以下两种情况：访问权限保护和存储区域保护。

1. 访问权限保护

访问权限保护就是看是否发生了访问越权。若实际访问操作与访问权限不符，则发生存储

保护错。通常通过在页表或段表中设置访问权限位来实现这种保护。一般规定：各程序对本程序所在的存储区可读可写；对共享区或已获授权的其他用户信息可读不可写；而对未获授权的信息（如 OS 内核、页表等）不可访问。通常，数据段可指定为可读可写或只读；程序段可指定只可执行或只读。

2. 存储区域保护

存储区域保护就是看是否发生了地址越界，也即是否访问了不该访问的区域。通常有以下几种常用的存储区域保护方式。

① 加界重定位。每个程序或程序段都记录有起始地址和终止地址，分别称为上界和下界。对虚拟地址加界（即加基准地址）生成物理地址后，如果物理地址超过了上界和下界规定的范围，则地址越界。有些系统用专门的一对上界寄存器和下界寄存器来记录上界和下界，在分段式虚存中，通过段表来记录段的上界和下界。

② 键保护。操作系统为主存的每一个页框分配一个存储键，为每个用户进程设置一个程序键。进程运行时，将程序状态字寄存器中的键（程序键）和所访问页的键（存储键）进行核对，相符时才可访问，这两个键如同“锁”与“钥匙”的关系。为使某个页框能被所有进程访问，或某个进程可访问任何一个页框，可规定键标志为 0，此时不进行核对工作。例如，操作系统有权访问所有页框中的页面，因此，可让内核进程的程序键为 0。

③ 环保护。主存中各进程按其重要性分为多个保护级，各级别构成同心环，最内环的进程保护级别最高，向外逐次降低。内环进程可以访问外环和同环进程的地址空间，而外环不得访问内环进程的地址空间。内核程序的保护级别最高，环号最小，而用户程序都处于外环上。IA-32 就采用该方案，操作系统内核工作在第 0 环（内核态），操作系统其他部分工作在第 1 环，用户进程工作在第 3 环（用户态），留下第 2 环给中间软件使用。实际上，Linux 等操作系统只用了第 0 环和第 3 环。

* 6.6 IA-32/Linux 中的地址转换

保护模式是 IA-32 微处理器最常用的工作模式。系统启动后总是先进入实地址模式，对系统进行初始化，然后才转入保护模式进行操作。这种工作模式提供了多任务环境下的各种复杂功能以及对存储器的虚拟管理机制。

在保护模式下，IA-32 采用段页式虚拟存储管理方式，存储空间采用逻辑地址、线性地址和物理地址来进行描述。逻辑地址就是通常所说的虚拟地址，IA-32 中的逻辑地址由 48 位组成，包含 16 位的段选择符和 32 位的段内偏移量（即有效地址）。为了便于多用户、多任务下的存储管理，IA-32 采用在分段基础上的分页机制。分段过程实现将逻辑地址转换为线性地址，分页过程再实现将线性地址转换为物理地址。

6.6.1 逻辑地址到线性地址的转换

为了说明逻辑地址到线性地址的转换过程，首先简要介绍段选择符、段描述符、段描述符表以及段描述符表寄存器等基本概念。

1. 段选择符和段寄存器

段选择符格式如图 6.36 所示, 其中 TI 表示段选择符选择哪一个段描述符表, 若 TI = 0, 表示选择全局描述符表 (GDT), 若 TI = 1, 表示选择局部描述符表 (LDT); RPL 用来定义当前程序段的特权等级, 若 RPL = 00, 则为第 0 级, 是最高级的内核态, 若 RPL = 11, 则为第 3 级, 是最低级的用户态; 高 13 位的索引值用来确定当前使用的段描述符在描述表中的位置, 表示是其中的第几个段表项。

15	14	...	3	2	1	0
索引				TI	RPL	

图 6.36 段选择符格式

段选择符存放在段寄存器中, 共有 6 个段寄存器: CS、SS、DS、ES、FS 和 GS。其中, 以下 3 个段寄存器具有专门的功能:

CS: 代码段寄存器, 指向程序代码所在的段。

SS: 栈段寄存器, 指向栈区所在的段。

DS: 数据段寄存器: 指向程序的全局静态数据区所在的段。

其他 3 个段寄存器可以指向任意的数据段。

CS 寄存器中的 RPL 字段表示 CPU 的当前特权级 (Current Privilege Level, CPL), Linux 只使用 0 级 (最高级) 和 3 级 (最低级), 分别为内核态和用户态。

2. 段描述符

段描述符是一种数据结构, 实际上就是分段方式下的段表项。根据段描述符的用途, 可以将其分为两种类型: 一种是普通的用户进程的代码段和数据段描述符, 另一种是系统控制段描述符。系统控制段描述符比较复杂, 按照不同的用途又可分为两种类型: 一种是特殊的系统控制段描述符, 包括局部描述符表描述符和任务状态段描述符; 另一种是控制转移描述符, 包括调用门描述符、任务门描述符、中断门描述符和陷阱门描述符。

一个段描述符占用 8 个字节, 其一般格式如图 6.37 所示, 包括 32 位的基地址 (B31 ~ B0)、20 位的界限 (L19 ~ L0) 和访问权限以及特征位 G、D 和 P 等, 其中 20 位的界限表示段中最大页号。

15					8	7	6	5	4	3	2	1	0	
7	基地址 (B31~B24)					G	D	0	AVL	限界 (L19~L16)				6
5	P	DPL	S	TYPE	A	基地址 (B23~B16)								4
3	基地址 (B15~B0)													2
1	限界 (L15~L0)													0

图 6.37 段描述符的一般格式

特征位的含义说明如下:

G: 表示粒度大小。G = 1 说明段以页 (4KB) 为基本单位; G = 0 则段以字节为基本单位。由于界限为 20 位, 所以当 G = 0 时, 最大的段为 $2^{20} \times 1\text{B} = 1\text{MB}$; 而当 G = 1 时, 最大的段为

$2^{20} \times 4\text{KB} = 4\text{GB}$ 。

D: D=1 表示段内偏移量为 32 位宽, D=0 表示段内偏移量为 16 位宽。

P: 说明段是否已存在主存中。P=1 表示存在, P=0 表示不存在。Linux 总是把 P 置 1, 因为它从来不会把一个段交换到磁盘上, 而是以页为单位交换的。

DPL: 访问段时对当前特权级的最低等级要求。因此, 只有 CPL 为 0 (内核态) 时才可访问 DPL 为 0 的段, 任何进程 (CPL=3 或 0) 都可以访问 DPL 为 3 的段。

S: S=0 表示是系统控制描述符, S=1 表示是普通的代码段或数据段描述符。

TYPE: 指示段的访问权限或系统控制描述符的类型。(通常包含字段 A。)

A: 说明段是否已被访问过。A=1 表示该段已被访问过, A=0 表示未被访问过。

AVL: 可以由操作系统定义使用。Linux 忽略该字段。

3. 描述符表

描述符表实际上就是分段方式下的段表, 由段描述符组成, 主要有三种类型: GDT (全局描述符表)、LDT (局部描述符表) 和 IDT (中断描述符表)。其中, GDT 只有一个, 用来存放系统内每个任务都可能访问的描述符, 例如, 后面提到的内核代码段、内核数据段、用户代码段、用户数据段以及 TSS (任务状态段) 等都属于 GDT 中描述的段; LDT 则是存放某一个任务 (即用户进程) 专用的描述符; IDT 则包含 256 个中断门、陷阱门和任务门描述符。有关 IDT 的详细说明将在第 7 章 7.2.5 节中介绍。

4. 用户不可见寄存器

为了支持 IA-32 的分段机制, 除了提供 6 个段寄存器外, 还提供了多个用户进程不可直接访问的内部寄存器, 它们包括描述符 cache、局部描述符表寄存器 (LDTR)、任务寄存器 (TR)、全局描述符表寄存器 (GDTR) 和中断描述符表寄存器 (IDTR), 如图 6.38 所示。

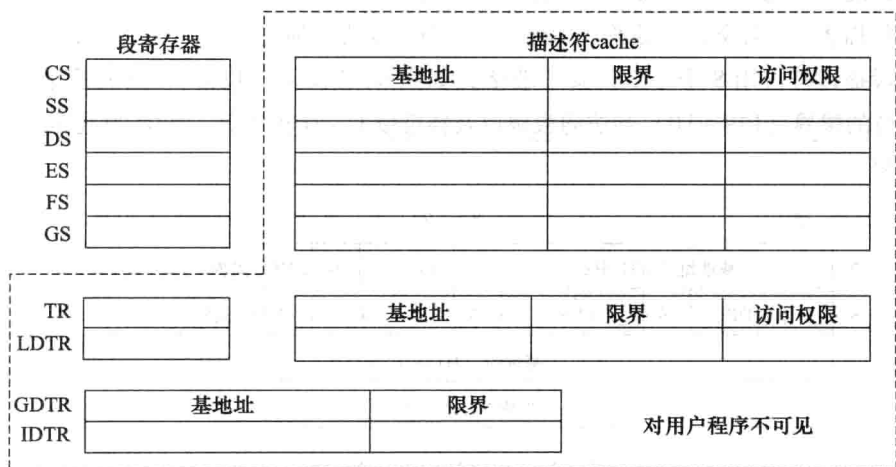


图 6.38 用户不可见寄存器

图 6.38 中虚线内的寄存器是用户程序感觉不到的, 因此称为用户不可见寄存器, 但是操作系统通过特权指令可对寄存器 TR、LDTR、GDTR 和 IDTR 进行读写。

描述符 cache 是一组用来存放当前段描述符信息的高速缓存，每当段寄存器装入新的段选择符时，处理器将段选择符指定的一个段描述符装入相应的描述符 cache 中。这样，在进行逻辑地址到线性地址的转换过程中，MMU 就直接用对应描述符 cache 中保存的基地址来形成线性地址 LA，而不必每次都去主存访问段表，从而大大节省访问存储器的时间。

每当段寄存器装入新的段选择符时，CPU 需要将段选择符指定的一个段描述符装入相应的描述符 cache 中，因此 CPU 需要知道描述符表的首地址。为此，在 CPU 内设置了相应的全局描述符表寄存器（GDTR）和中断描述符表寄存器（IDTR）。GDTR 和 IDTR 的高 32 位分别存放 GDT 首地址和 IDT 首地址，低 16 位存放限界，即最大字节数，因而两个描述符表 GDT 和 IDT 的最大长度为 $2^{16}\text{B} = 64\text{KB}$ 。

局部描述符表寄存器（LDTR）是 16 位寄存器，存放局部描述符表的选择符，通过该选择符可把在全局描述符表中的 LDT 描述符（包含 LDT 首地址、LDT 界限和访问权限等）装入 LDT 描述符 cache 中，从而使 CPU 可以快速访问 LDT。任务寄存器（TR）也是 16 位，用来存放 任务状态段（TSS）的选择符。通过该选择符可把全局描述符表中的 TSS 描述符（包含 TSS 首地址、TSS 界限和访问权限等）装入 TSS 描述符 cache 中，从而可以方便地对 任务（即用户进程）进行控制。

5. 逻辑地址向线性地址的转换

逻辑地址向线性地址的转换过程如图 6.39 所示。

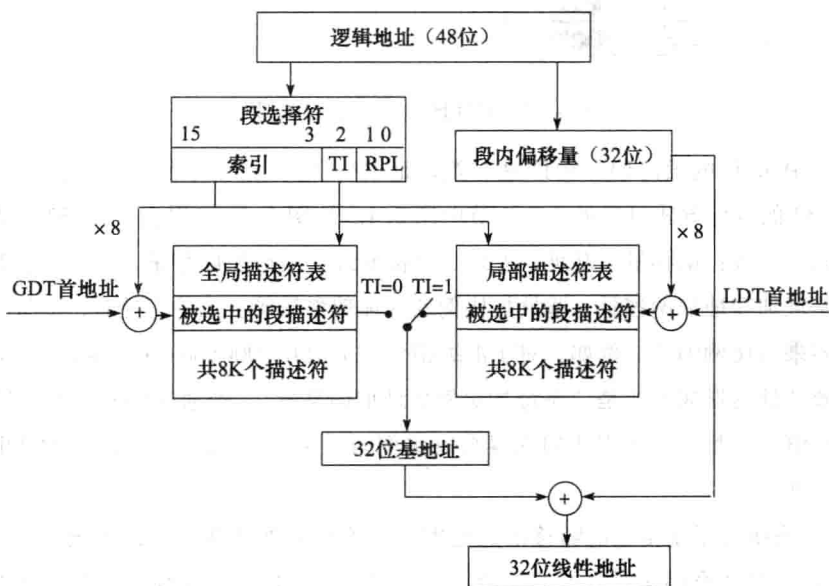


图 6.39 逻辑地址向线性地址转换的过程

逻辑地址包含 16 位的段选择符和 32 位的段内偏移量。如图 6.39 所示，MMU 首先根据段选择符中的 TI 确定选择全局描述符表（GDT）还是局部描述符表（LDT）。若 $TI = 0$ ，选用 GDT；否则，选用 LDT。确定描述符表后，再通过段选择符内的 13 位索引值，从被选中的描述符表中找到对应的段描述符。因为每个段描述符占 8 个字节，所以位移量为索引值乘 8，加上

描述符表首地址（其中，GDT 首地址从 GDTR 的高 32 位获得，LDT 首地址从 LDTR 对应的 LDT 描述符 cache 中高 32 位获得），就可以确定选中的段描述符的地址，从中取出 32 位的基地址（B31 ~ B0），与逻辑地址中 32 位的段内偏移量相加，就得到 32 位线性地址。MMU 在计算线性地址 LA 的过程中，可以根据段的限界和段的访问权限判断是否“地址越界”或“访问越权”，以实现存储保护。

通常情况下，MMU 并不需要到主存中去访问 GDT 或 LDT，而只要根据如图 6.38 所示的段寄存器对应的描述符 cache 中的基地址、限界和访问（存取）权限来进行逻辑地址到线性地址的转换，如图 6.40 所示。

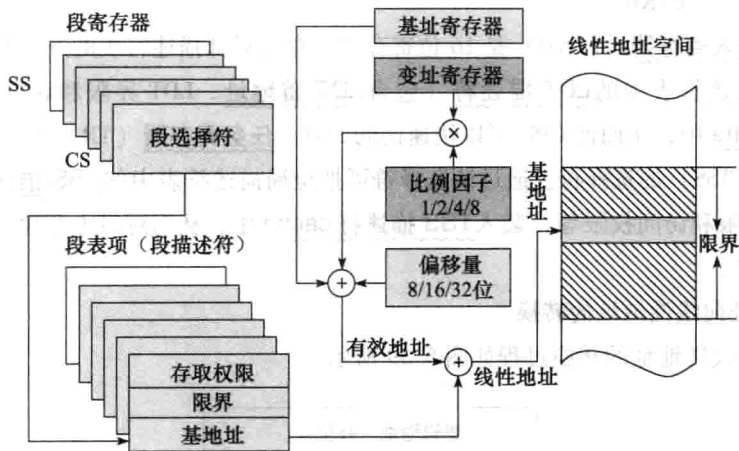


图 6.40 线性地址 LA 的形成过程

逻辑地址中 32 位的段内偏移量即是有效地址 EA，它由指令中的寻址方式来确定如何得到，有关 IA-32 的寻址方式可参见 3.2.2 节中图 3.4。从图 6.40 可看出，IA-32 中有效地址的形成方式有以下几种：偏移量、基址、变址、比例变址、基址加偏移量、基址加变址、基址加比例变址、基址加变址加偏移量、基址加比例变址加偏移量等。比例变址时，变址值等于变址寄存器的内容乘以比例因子。例如，对于汇编指令“movl 0x1000(%ebp,%esi,4),%eax”，其源操作数的有效地址的形成方式是“基址加比例变址加偏移量”，即通过将基址寄存器 EBP 的内容、比例变址值（变址寄存器 ESI 的内容乘以比例因子 4）、偏移量（即位移 1000H）三者相加得到有效地址。

Linux 操作系统为了使得它能够移植到绝大多数流行的处理器平台，简化了段页式虚拟存储管理。因为 RISC 体系结构对分段的支持非常有限，所以 Linux 仅使用了 IA-32 架构中的分页机制，而对于分段机制，则通过在初始化时将所有段描述符的基地址全部设为 0 来简化其功能。

我们把运行在用户态的所有 Linux 进程使用的代码段和数据段分别称为用户代码段和用户数据段；把运行在内核态的所有 Linux 进程使用的代码段和数据段分别称为内核代码段和内核数据段。Linux 初始化时，将上述 4 个段的段描述符中各字段设置成表 6.2 中的值。

表 6.2 Linux 中设置的 4 个段描述符中各字段的内容

段	基地址	G	限界	S	TYPE	DPL	D	P
用户代码段	0x0000 0000	1	0xFFFFF	1	10	3	1	1
用户数据段	0x0000 0000	1	0xFFFFF	1	2	3	1	1
内核代码段	0x0000 0000	1	0xFFFFF	1	10	0	1	1
内核数据段	0x0000 0000	1	0xFFFFF	1	2	0	1	1

注：表中 TYPE 字段包含 A 字段，因而占 4 位。

从表 6.2 可以看出，用户态和内核态所有进程的每个段的线性地址都是从基地址 0 开始，都是以 4KB 为粒度 ($G=1$) 计算最大段内地址，因而最大段内地址为 $4K \times 0xFFFFF = 2^{32} - 1$ ，也即每个段的线性地址空间大小都是 4GB，与逻辑地址中的段内偏移量形成的空间大小一样。在 Linux 系统中，因为所有代码段和数据段的基地址都为 0，所以，所有逻辑地址中的段内偏移量（即有效地址）就是其线性地址。

例 6.8 已知变量 y 和数组 a 都是 int 型， a 的首地址为 $0x8048a00$ 。假设编译器将 a 的首地址分配在 ECX 中，数组的下标变量 i 分配在 EDX 中， y 分配在 EAX 中，C 语言赋值语句 “ $y = a[i];$ ” 被编译为指令 “ $\text{movl}(\%ecx, \%edx, 4), \%eax$ ”。若在 IA-32/Linux 环境下执行指令地址为 $0x80483c8$ 的该指令时，CS 段寄存器对应的描述符 cache 中存放的是表 6.2 中所示的用户代码段信息且 $\text{CPL}=3$ ，DS 段寄存器对应的描述符 cache 中存放的是表 6.2 中所示的用户数据段信息，则当 $i=100$ 时，取指令操作过程中 MMU 得到的指令的线性地址是多少？取数操作过程中 MMU 得到的操作数的线性地址是多少？

解 IA-32 执行指令 “ $\text{movl}(\%ecx, \%edx, 4), \%eax$ ” 需两次存储访问操作，一次是取指令操作，一次是取数操作，在保护模式下每次存储访问操作 MMU 都要对访存地址进行逻辑地址到线性地址的转换。

在取指令操作中，MMU 先根据 CS 对应的段描述符获得 DPL，确定 DPL 的特权级别不高于 CPL 才能继续进行地址转换，否则发生存储保护错。显然，这里 $\text{DPL}=\text{CPL}=3$ ，没有发生存储保护错，于是，MMU 将 CS 对应的段描述符中的基地址与指令地址（指令地址即为指令在代码段的段内偏移量）相加，得到线性地址为 $0x0 + 0x80483c8 = 0x80483c8$ 。

在取数操作中，MMU 先根据 DS 对应的段描述符获得 DPL，确定 DPL 的特权级别不高于 CPL 才能继续进行地址转换，否则发生存储保护错。显然，这里 $\text{DPL}=\text{CPL}=3$ ，没有发生存储保护错，于是，MMU 将 DS 对应的段描述符中的基地址与操作数有效地址相加得到线性地址。因为操作数 “ $(\%ecx, \%edx, 4)$ ” 的寻址方式为 “基址加比例变址加偏移量”，故有效地址 $\text{EA} = \text{R}[\text{ecx}] + \text{R}[\text{edx}] \times 4 + 0 = 0x8048a00 + 100 \times 4 = 0x8048b90$ 。因此，操作数的线性地址为 $0x0 + 0x8048b90 = 0x8048b90$ 。 ■

6.6.2 线性地址到物理地址的转换

IA-32 采用段页式虚拟存储管理方式，通过分段方式完成逻辑地址到线性地址的转换后，再进一步通过分页方式将线性地址转换为物理地址。IA-32 内部有多个 32 位控制寄存器，它们与分页阶段的地址转换过程相关，因此，在介绍线性地址到物理地址的转换之前，先介绍

IA-32 中的控制寄存器。

1. 控制寄存器

控制寄存器保存了机器的各种控制和状态信息，这些控制和状态信息将影响系统所有任务的运行，操作系统进行任务控制或存储管理时将使用这些控制和状态信息。主要的几个控制寄存器以及存放的控制、状态信息说明如下。

CR0 控制寄存器定义了多个控制位。① 保护模式允许位 PE。用来确定处理器工作于实模式还是保护模式。系统启动时 $PE=0$ ，处于实模式。可用 MOV 指令将 PE 置 1，使机器进入保护模式。一旦进入保护模式，处理器不能再由软件将 PE 清 0，只能重启系统以回到实模式。这个标志仅能开启段保护，并没有启用分页机制。② 分页允许位 PG。若 $PG=1$ ，则启用分页部件工作；若 $PG=0$ 则禁止分页部件工作，此时线性地址被直接作为物理地址使用。若要启用分页机制，那么 PE 和 PG 标志都要置 1。③ 任务切换位 TS。每次任务切换时将其置 1，任务切换完毕则清 0，可用 CLTS 指令将其清 0。④ 对齐屏蔽位 AM。它可与 EFLAGS 中的 AC 位配合使用。若 $AM=1$ 且 $AC=1$ ，则进行对齐检查；若 $AM=0$ ，则禁止对齐检查。⑤ cache 功能控制位 NW ((Not Write-through) 和 CD (Cache Disable)。只有当 NW 和 CD 均为 0 时，cache 才能工作。

CR2 是页故障线性地址寄存器，存放引起页故障（即缺页）的线性地址。只有在 CR0 中的 $PG=1$ 时，CR2 才有效。当页故障处理程序（也称缺页异常处理程序）被激活时，压入对应栈中的错误码将提供页故障的状态信息，页故障处理程序根据错误码进行不同的对应处理，有关内容参见第 7 章。

CR3 是页目录基址寄存器，用来保存页目录表的起始地址。只有当 CR0 中的 $PG=1$ 时，CR3 才有效。

2. 线性地址向物理地址转换

图 6.41 所示的是分页部件将线性地址转换为物理地址的基本过程，为了解决页表过大的问题，采用了两级页表方式。

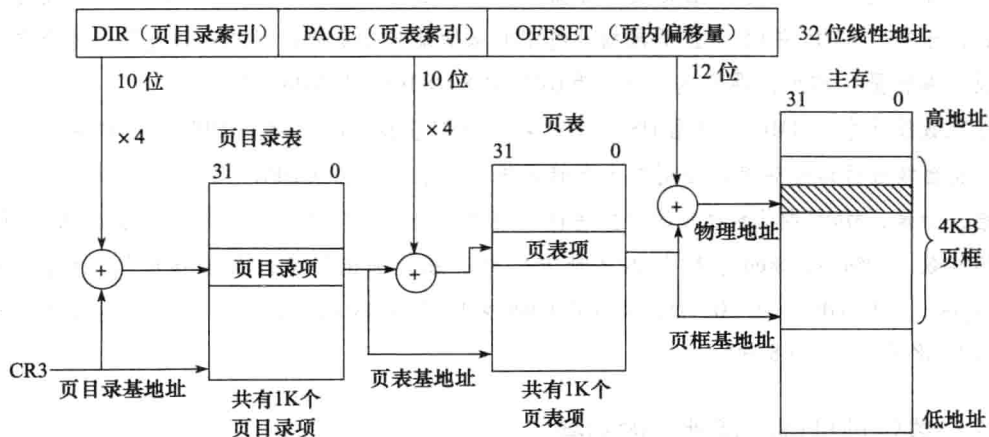


图 6.41 线性地址向物理地址转换的过程

从图 6.41 可看出，在一个两级页表分页方式中，线性地址由三个字段组成，它们分别是 10 位页目录索引 (DIR)、10 位页表索引 (PAGE) 和 12 位页内偏移量 (OFFSET)。

页目录项和页表项的格式如图 6.42 所示。

31	12	11	10	9	8	7	6	5	4	3	2	1	0
基地址				未使用	0	0	D	A	PCD	PWT	U/S	R/W	P

图 6.42 页目录项和页表项的格式

页目录项和页表项中部分字段的含义简述如下。

P: $P=1$ 表示页表或页在主存中; $P=0$ 表示页表或页不在主存中, 此时发生页故障 (即缺页异常), 需将页故障线性地址记录在 CR2 中。操作系统在处理页故障时会将缺失的页表或页从磁盘装入主存中, 并重新执行引起页故障的指令。

R/W: 该位为 0 时表示页表或页只能读不能写; 为 1 时表示可读可写。

U/S: 该位为 0 时表示用户进程不能访问; 为 1 时允许用户进程访问。该位可以保护操作系统所使用的页不受用户进程的破坏。

PWT: 用来控制页表或页对应的 cache 写策略是全写 (write through) 还是回写 (write back)。

PCD: 用来控制页表或页能否被缓存到 cache 中。

A: $A=1$ 表示指定页表或页被访问过, 初始化时操作系统将其清 0。利用该标志, 操作系统可清楚地了解哪些页表或页正在使用, 一般选择长期未用的页或近来最少使用的页调出主存。由 MMU 在进行地址转换时将该位置 1。

D: 修改位或称脏位 (dirty bit)。该位在页目录项中没有意义, 只在页表项中有意义。 $D=1$ 表示页被修改过; 否则说明页面内容未被修改, 因而在操作系统将页面替换出主存时, 无需将页面写入磁盘。初始化时操作系统将其清 0, 由 MMU 在进行写操作的地址转换时将该位置 1。

页目录项和页表项中的高 20 位是页表或页在主存中的首地址对应的页框号, 即首地址的高 20 位。每个页表的起始位置都按 4KB 对齐。

从图 6.41 可看出, 线性地址向物理地址的转换过程如下: 首先, 根据控制寄存器 CR3 中给出的页目录表首地址找到页目录表, 由 DIR 字段提供的 10 位页目录索引找到对应的页目录项, 每个页目录项大小为 4B; 然后, 根据页目录项中 20 位基地址指出的页表首地址找到对应页表, 再根据线性地址中间的页表索引 (PAGE 字段) 找到页表中的页表项; 最后, 将页表项中的 20 位基地址和线性地址中的 12 位页内偏移量组合成 32 位物理地址。上述转换过程中, 10 位的页目录索引和 10 位的页表索引都要乘以 4, 因为每个页目录项和页表项都是 32 位, 占 4 个字节。

由于页目录索引和页表索引均为 10 位, 每个页目录项和页表项占用 4 个字节, 因此页目录表和页表的长度均为 4KB, 并分别含有 1024 个表项。这样, 对于 12 位偏移地址, 32 位的线性地址所映射的物理地址空间是 $1024 \times 1024 \times 4\text{KB} = 4\text{GB}$ 。

如果线性地址空间更大的话, 可以将上述两级页表方式进一步扩展为 3 级页表或 4 级页表等方式。例如, 在 Intel Core i7 中就采用了 4 级页表方式。

6.7 小结

本章主要包括存储器的分类、存储器分层结构、半导体存储器、主存储器与 CPU 的连接、磁盘存储器、高速缓冲存储器 cache 的基本原理、cache 和主存之间的地址映射、替换算法、虚拟存储器的基本概念、页表结构、缺页异常、后备转换缓冲器 (TLB) 等。

因为每一类单独的存储器都不可能又快、又大、又便宜,为了构建理想的存储器系统,计算机内部采用层次化存储器体系结构。按照速度从快到慢、容量从小到大、价格从贵到便宜、与 CPU 连接的距离由近到远的顺序,将不同类型的存储器设置在计算机中,其设置的顺序为寄存器→cache→主存→磁盘→光盘和磁带。

利用程序访问的局部性特点,通常把主存中的一块数据复制到靠近 CPU 的 cache 中。cache 和主存间的映射有直接映射、全相联映射和组相联映射。替换算法主要有 FIFO 和 LRU。写策略有回写法和全写法。

虚拟存储机制的引入,使得每个进程具有一个一致的、极大的、私有的虚拟地址空间。虚拟地址空间按等长的页来划分,主存也按等长的页框划分。进程执行时将当前用到的页面装入主存,其他暂时不用的部分放在磁盘上,通过页表建立虚拟页和主存页框之间的对应关系。不在主存的页面在页表中记录其在磁盘上的地址,在指令执行过程中,由特殊硬件 (MMU) 和操作系统一起实现存储访问。虚拟存储器有分页式、分段式、段页式三类。虚拟地址需转换成物理地址。每个进程有一个页表,每个页表项由有效 (装入) 位、使用位、修改位、访问权限位、禁止缓存位、存放位置字段 (主存页框号或磁盘地址) 等组成。为减少访问内存中页表的次数,通常将活跃页的页表项放到一个特殊的高速缓存 TLB (快表) 中。虚拟存储机制能实现存储保护,通常有地址越界和访问越权两种存储保护错。

习题

1. 给出以下概念的解释说明。

随机存取存储器 (RAM)	顺序存取存储器 (SAM)	直接存取存储器 (DAM)
相联存储器 (AM)	只读存储器 (ROM)	读写存储器
易失性存储器	记忆单元 (cell)	存储阵列 (bank)
编址单位	编址方式	最大可寻址范围
读出时间	写入时间	存储周期
存储器带宽	静态 RAM (SRAM)	动态 RAM (DRAM)
闪存 (Flash 存储器)	行地址选通信号 (RAS)	列地址选通信号 (CAS)
超元 (supercell)	内部行缓冲	磁盘驱动器
磁盘控制器	未格式化容量	格式化容量
寻道时间	旋转等待时间	数据传输时间
平均存取时间	固态硬盘 (SSD)	多模块交叉存储

高速缓存 (cache)	时间局部性	空间局部性
主存块	cache 行 (槽)	命中率
命中时间	缺失率	缺失损失
直接映射	全相联映射	组相联映射
替换策略	FIFO 算法	LRU 算法
LRU 位	cache 一致性	回写法 (write back)
全写法 (write through)	关联度	虚拟地址 (逻辑地址)
虚拟页号	物理地址	页框 (页帧)
物理页号	未分配页	已分配页
未缓存页	已缓存页	请求分页
页故障 (page fault)	页表	页表基址寄存器
有效位 (装入位)	修改位 (脏位)	访问权限位
快表 (TLB)	段表	段页式
特权指令	特权模式	内核态 (模式)
用户态 (模式)	存储保护	地址越界
访问越权		

2. 简单回答下列问题。

- (1) 计算机内部为何要采用层次结构存储体系? 层次结构存储体系如何构成?
- (2) SRAM 芯片和 DRAM 芯片各有哪些特点? 各自用在哪些场合?
- (3) 为什么采用地址对齐方式能减少数据访问 DRAM 的时间?
- (4) 为什么在 CPU 和主存之间引入 cache 能提高 CPU 的访存效率?
- (5) 为什么说 cache 对程序员来说是透明的?
- (6) 什么是 cache 映射的关联度? 关联度与命中率、命中时间的关系各是什么?
- (7) 为什么直接映射方式不需要考虑替换策略?
- (8) 为什么要考虑 cache 的一致性? 读操作时是否要考虑 cache 的一致性问题? 为什么?
- (9) 什么是物理地址? 什么是逻辑地址? 地址转换由硬件还是软件实现? 为什么?
- (10) 什么是页表? 什么是快表 (TLB)?
- (11) 在存储器层次化结构中, “cache—主存”、“主存—磁盘”这两个层次有哪些不同?

3. 某计算机主存最大寻址空间为 64KB, 按字节编址, 假定用 $1K \times 8$ 位的 DRAM 芯片组成容量为 16KB 的内存条 (主存模块)。回答下列问题。

- (1) 每个内存条需要多少 DRAM 芯片?
- (2) 构建容量为 48KB 的主存时, 需要几个内存条?
- (3) 主存地址共有多少位? 其中哪几位用作 DRAM 芯片内地址? 哪几位为 DRAM 芯片内的行地址? 哪几位为 DRAM 芯片内的列地址?

4. 某计算机按字节编址, 其中已配有 0000H ~ 7FFFH 的 ROM 区域, 现在再用 $16K \times 4$ 位的 RAM 芯片形成 $32K \times 8$ 位的存储区域, CPU 地址线为 A0 ~ A15。回答下列问题。

- (1) RAM 区的地址范围是什么? 共需要多少 RAM 芯片? 地址线中哪一位用来区分 ROM 区和 RAM 区?
 - (2) 假定 CPU 地址线改为 24 根, 地址范围 000000H ~ 007FFFH 为 ROM 区, 剩下的所有地址空间都用 16K × 4 位的 RAM 芯片配置, 则需要多少个这样的 RAM 芯片?
5. 假设一个程序重复完成将磁盘上一个 4KB 的数据块读出, 进行相应处理后, 写回到磁盘的另外一个数据区。各数据块内信息在磁盘上连续存放, 数据块随机位于磁盘的一个磁道上。磁盘转速为 7200 转/分, 平均寻道时间为 10ms, 磁盘最大内部数据传输率为 40MB/s, 磁盘控制器的开销为 2ms, 没有其他程序使用磁盘和处理器, 并且磁盘读写操作和磁盘数据的处理时间不重叠。若程序对磁盘数据的处理需要 20 000 个时钟周期, 处理器时钟频率为 500MHz, 则该程序完成一次数据块“读出一处理一写回”操作所需的时间为多少? 每秒钟可以完成多少次这样的数据块操作?
 6. 现代计算机中, SRAM 一般用于实现快速小容量的 cache, 而 DRAM 用于实现慢速大容量的主存。以前超级计算机通常不提供 cache, 而是用 SRAM 来实现主存 (如, Cray 巨型机), 请问: 如果不考虑成本, 你还这样设计高性能计算机吗? 为什么?
 7. 对于数据的访问, 分别给出具有下列要求的程序或程序段的示例。
 - (1) 几乎没有时间局部性和空间局部性。
 - (2) 有很好的时间局部性, 但几乎没有空间局部性。
 - (3) 有很好的空间局部性, 但几乎没有时间局部性。
 - (4) 空间局部性和时间局部性都好。
 8. 假设某计算机主存地址空间大小为 1GB, 按字节编址, cache 的数据区 (即不包括标记、有效位等存储区) 有 64KB, 块大小为 128B, 采用直接映射和全写 (write-through) 方式。回答下列问题。
 - (1) 主存地址多少位? 如何划分? 要求说明每个字段的含义、位数和在主存地址中的位置。
 - (2) cache 的总容量为多少位?
 9. 假设某计算机的 cache 共 16 行, 开始为空, 主存块大小为 1 个字, 采用直接映射方式, 按字编址。CPU 执行某程序时, 依次访问以下地址序列: 2, 3, 11, 16, 21, 13, 64, 48, 19, 11, 3, 22, 4, 27, 6 和 11。回答下列问题。
 - (1) 访问上述地址序列得到的命中率是多少?
 - (2) 若 cache 数据区容量不变, 而主存块大小改为 4 个字, 则上述地址序列的命中情况又如何?
 10. 假设数组元素在主存按从左到右的下标顺序存放, N 是用 #define 定义的常量。试改变下列函数中循环的顺序, 使得其数组元素的访问与排列顺序一致, 并说明为什么在 N 较大的情况下修改后的程序比原来的程序执行时间更短。

```
int sum_array (int a[N][N][N])
{
    int i, j, k, sum=0;
    for (i=0; i < N; i++)
        for (j=0; j < N; j++)
            for (k=0; k < N; k++) sum += a[k][i][j];
    return sum;
}
```


11. 分析比较以下三个函数中数组访问的空间局部性, 并指出哪个最好, 哪个最差?

<pre># define N 1000 typedef struct { int vel[3]; int acc[3]; } point; point p[N]; void clear1(point *p, int n) { int i, j; for (i = 0; i < n; i++) { for (j = 0; j < 3; j++) p[i].vel[j] = 0; for (j = 0; j < 3; j++) p[i].acc[j] = 0; } }</pre>	<pre># define N 1000 typedef struct { int vel[3]; int acc[3]; } point; point p[N]; void clear2(point *p, int n) { int i, j; for (i=0; i<n; i++) { for (j=0; j<3; j++) { p[i].vel[j] = 0; p[i].acc[j] = 0; } } }</pre>	<pre># define N 1000 typedef struct { int vel[3]; int acc[3]; } point; point p[N]; void clear3(point *p, int n) { int i, j; for (j=0; j<3; j++) { for (i=0; i<n; i++) p[i].vel[j] = 0; for (i=0; i<n; i++) p[i].acc[j] = 0; } }</pre>
--	---	--

12. 以下是计算两个向量点积的程序段:

```
float dotproduct (float x[8], float y[8])
{
    float sum = 0.0;
    int i;;
    for (i = 0; i < 8; i++) sum += x[i] * y[i];
    return sum;
}
```

回答下列问题或完成下列任务。

- (1) 试分析该段代码中访问数组 x 和 y 的时间局部性和空间局部性, 并推断命中率的高低。
- (2) 假设该段程序运行的计算机中的数据 cache 采用直接映射方式, 其数据区容量为 32B, 主存块大小为 16B; 编译程序将变量 sum 和 i 分配给寄存器, 数组 x 存放在 0x8049040 开始的主存区域, 数组 y 紧跟在 x 后。试计算该程序中数据访问的命中率, 要求说明每次访问时 cache 的命中情况。
- (3) 将 (2) 中的数据 cache 改用 2 路组相联映射方式, 主存块大小改为 8B, 其他条件不变, 则该程序数据访问的命中率是多少?
- (4) 在 (2) 中条件不变的情况下, 将数组 x 定义为 $\text{float}[12]$, 则数据访问的命中率又是多少?

13. 以下是对矩阵进行转置的程序段:

```
typedef int array[4][4];
void transpose(array dst, array src)
{
    int i, j;
    for (i = 0; i < 4; i++)
        for (j = 0; j < 4; j++) dst[j][i] = src[i][j];
}
```

假设该段程序运行的计算机中 `sizeof(int) = 4`，且只有一级 cache，其中 L1 数据 cache 的数据区大小为 32B，采用直接映射、回写方式，块大小为 16B，初始为空。数组 `dst` 从地址 0x804c000 开始存放，数组 `src` 从地址 0x804c010H 开始存放。仿照例子填写下表，说明数组元素 `src[row][col]` 和 `dst[row][col]` 各自映射到 cache 哪一行，访问是命中 (hit) 还是缺失 (miss)。若 L1 数据 cache 的数据区容量改为 128B 时，重新填写下表中的内容。

	src 数组				dst 数组			
	col = 0	col = 1	col = 2	col = 3	col = 0	col = 1	col = 2	col = 3
row = 0	0/miss							
row = 1								
row = 2								
row = 3								

14. 假设某计算机的主存地址空间大小为 64MB，按字节编址。其 cache 数据区容量为 4KB，采用 4 路组相联映射、LRU 替换算法和回写 (write back) 策略，块大小为 64B。请问：
- (1) 主存地址字段如何划分？要求说明每个字段的含义、位数和在主存地址中的位置。
 - (2) 该 cache 的总容量有多少位？
 - (3) 假设 cache 初始为空，CPU 依次从 0 号地址单元顺序访问到 4344 号单元，重复按此序列共访问 16 次。若 cache 命中时间为 1 个时钟周期，缺失损失为 10 个时钟周期，则 CPU 访存的平均时间为多少时钟周期？
15. 假定某处理器可通过软件对高速缓存设置不同的写策略，那么，在下列两种情况下，应分别设置成什么写策略？为什么？
- (1) 处理器主要运行包含大量存储器写操作的数据访问密集型应用。
 - (2) 处理器运行程序的性质与 (1) 相同，但安全性要求很高，不允许有任何数据不一致的情况发生。
16. 已知 cache 1 采用直接映射方式，共 16 行，块大小为 1 个字，缺失损失为 8 个时钟周期；cache 2 也采用直接映射方式，共 4 行，块大小为 4 个字，缺失损失为 11 个时钟周期。假定开始时 cache 为空，采用字编址方式。要求找出一个访问地址序列，使得 cache 2 具有更低的缺失率，但总的缺失损失反而比 cache 1 大。
17. 提高关联度通常会降低缺失率，但并不总是这样。请给出一个地址访问序列，使得采用 LRU 替换算法的 2 路组相联映射 cache 比具有同样大小的直接映射 cache 的缺失率更高。
18. 假定有三个处理器，分别带有以下不同的 cache。
- cache 1：采用直接映射方式，块大小为 1 个字，指令和数据的缺失率分别为 4% 和 6%；
 - cache 2：采用直接映射方式，块大小为 4 个字，指令和数据的缺失率分别为 2% 和 4%；
 - cache 3：采用 2 路组相联映射方式，块大小为 4 个字，指令和数据的缺失率分别为 2% 和 3%。
- 在这些处理器上运行同一个程序，其中有一半是访存指令，在三个处理器上测得该程序的 CPI 都为 2.0。已知处理器 1 和 2 的时钟周期都为 420ps，处理器 3 的时钟周期为 450ps。若缺失损失为 (块大小 + 6) 个时钟周期，请问：哪个处理器因 cache 缺失而引起的额外开销最大？哪个处理器执行速度最快？
19. 假定某处理器带有一个数据区容量为 256B 的 cache，其主存块大小为 32B。以下 C 语言程序段运行在该处理器上，设 `sizeof(int) = 4`，编译器将变量 `i`、`j`、`c`、`s` 都分配在通用寄存器

中, 因此, 只要考虑数组元素的访存情况。为简化问题, 假定数组 a 从一个主存块开始处存放。若 cache 采用直接映射方式, 则当 $s=64$ 和 $s=63$ 时, 缺失率分别为多少? 若 cache 采用 2 路组相联映射方式, 则当 $s=64$ 和 $s=63$ 时, 缺失率又分别为多少?

```
int i, j, c, s, a[128];
.....
for (i = 0; i < 10000; i++)
    for (j = 0; j < 128; j=j+s)
        c = a[j];
```

20. 假定一个虚拟存储系统的虚拟地址为 40 位, 物理地址为 36 位, 页大小为 16KB。若页表中有有效位、访问权限位、修改位、使用位, 共占 4 位, 磁盘地址不记录在页表中, 则该存储系统中每个进程的页表大小为多少? 如果按计算出来的实际大小构建页表, 则会出现什么问题?
21. 假定一个计算机系统有一个 TLB 和一个 L1 数据 cache。该系统按字节编址, 虚拟地址 16 位, 物理地址 12 位, 页大小为 128B; TLB 采用 4 路组相联方式, 共有 16 个页表项; L1 数据 cache 采用直接映射方式, 块大小为 4B, 共 16 行。在系统运行到某一时刻时, TLB、页表和 L1 数据 cache 中的部分内容如下表所示:

a) TLB (4路组相联): 4组、16个页表项

组号	标记	页框号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号	有效位
0	03	-	0	09	0D	1	00	-	0	07	02	1
1	13	2D	1	02	-	0	04	-	0	0A	-	0
2	02	-	0	08	-	0	06	-	0	03	-	0
3	07	-	0	63	0D	1	0A	34	1	72	-	0

b) 部分页表 (开始16项)

虚页号	页框号	有效位
00	08	1
01	03	1
02	14	1
03	02	1
04	-	0
05	16	1
06	-	0
07	07	1
08	13	1
09	17	1
0A	09	1
0B	-	0
0C	19	1
0D	-	0
0E	11	1
0F	0D	1

c) L1数据cache: 直接映射, 共16行, 块大小为4B

行索引	标记	有效位	字节3	字节2	字节1	字节0
0	19	1	12	56	C9	AC
1	-	0	-	-	-	-
2	1B	1	03	45	12	CD
3	-	0	-	-	-	-
4	32	1	23	34	C2	2A
5	0D	1	46	67	23	3D
6	-	0	-	-	-	-
7	16	1	12	54	65	DC
8	24	1	23	62	12	3A
9	-	0	-	-	-	-
A	2D	1	43	62	23	C3
B	-	0	-	-	-	-
C	12	1	76	83	21	35
D	16	1	A3	F4	23	11
E	33	1	2D	4A	45	55
F	-	0	-	-	-	-

请问 (假定图中数据都为十六进制形式):

- (1) 虚拟地址中哪几位表示虚拟页号? 哪几位表示页内偏移量? 虚拟页号中哪几位表示 TLB 标记? 哪几位表示 TLB 组索引?

- (2) 物理地址中哪几位表示物理页号？哪几位表示页内偏移量？
- (3) 主存物理地址如何划分成标记字段、行索引字段和块内地址字段？
- (4) CPU 从虚拟地址 067AH 中取出的值为多少？说明 CPU 读取地址 067AH 中内容的过程。
22. 对于第 3 章 3.6 节所介绍的缓冲区溢出漏洞，你认为可以采用本章提到的什么技术来防止。
23. 假设在 IA-32/Linux 平台上运行一个 C 语言源程序 P 对应的用户进程，P 中有一条循环语句 S 如下：

```
for (i=0; i<N; i++) sum += a[i];
```

已知变量 *sum* 和数组 *a* 都是 long 型，链接后确定 *a* 的首地址为 0x804d000。假设编译器将 *a* 的首地址分配在 EDI 中，数组的下标变量 *i* 分配在 ECX 中，*sum* 分配在 EAX 中，赋值语句“*sum* += *a*[*i*];”仅用一条指令 I 实现。已知 IA-32/Linux 平台采用图 6.41 所示的两级页表分页虚拟存储管理方式，页大小为 4KB，系统启动后控制寄存器 CR0 中的控制位 NW 和 CD 均为 0。假定系统中没有其他用户进程，回答下列问题或完成下列任务。

- (1) 指令 I 对应的汇编形式 (AT&T 格式) 是什么？指令 I 中存储器操作数的寻址方式是哪种？
- (2) 假定指令 I 的地址为 0x8049c08，执行指令 I 时，CS 段寄存器对应的描述符 cache 中存放的是表 6.2 中所示的用户代码段信息且 CPL=3，DS 段寄存器对应的描述符 cache 中存放的是表 6.2 中所示的用户数据段信息，则当 *i*=50 时，取指令操作过程中 MMU 得到的指令的线性地址是多少？取数操作过程中 MMU 得到的操作数的线性地址是多少？
- (3) 假定 *N* 在 EBX 中，写出语句 S 对应的指令序列，与 GCC 生成的代码进行比较。
- (4) 在执行到程序 P 时，控制寄存器 CR0 中的控制位 PE 和 PG 各是什么？
- (5) 指令 I 在第一次执行过程中，有没有可能发生缺页异常？为什么？如果发生缺页异常的话，则页故障线性地址是什么？该地址会保存在哪个控制寄存器中？
- (6) 指令 I 所在页的虚页号是什么？指令 I 的线性地址中，页目录索引、页表索引和页内偏移量分别是什么？第一次执行指令 I 时，指令 I 所在页对应页表项中，字段 P、R/W、U/S、A 和 D 的内容各是什么？
- (7) 指令 I 在第一次执行过程中，有没有可能发生 TLB 缺失？为什么？若指令 TLB 共有 16 个表项，采用 4 路组相联方式，则虚页号中哪几位为 TLB 标记？哪几位表示 TLB 组索引？若第一次执行到指令 I 时，指令 TLB 中的部分内容如下表所示 (TLB 表项中部分字段缺失，内容以十六进制表示)，则指令 I 所存放的主存地址是什么？

组号	标记	页框号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号	有效位
0	03010	00A10	1	02101	0D001	1	00000	00000	0	00000	00000	0
1	02011	02D02	1	02012	028B0	1	04001	02012	1	00000	00000	0
2	02001	08902	1	02120	09200	0	02010	0340A	0	02301	0320A	1
3	01002	08770	1	00000	00000	0	0A001	02010	1	00000	00000	0

- (8) 若指令 cache 的数据区容量为 8KB，主存块大小为 32B，采用 2 路组相联映射方式，则指令 I 在第一次执行时的取指令过程中，会不会发生 cache 缺失？指令 I 所在的主存块应映射到指令 cache 的哪一组中？
- (9) 当 *N*=2000 时，数组 *a* 占用几个页面？每个页的虚页号是什么？数组元素 *a*[1200] 在哪个页中？